

Propose, then judge: a small calibrated decision model versus large language model judges for multilingual slot filling

Christophe Cruz, Hussam Ghanem, Hocine Cherifi

▶ **To cite this version:**

Christophe Cruz, Hussam Ghanem, Hocine Cherifi. Propose, then judge: a small calibrated decision model versus large language model judges for multilingual slot filling. 2026. ⟨hal-05850913⟩

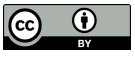
HAL Id: hal-05850913

https://hal.science/hal-05850913v1

Preprint submitted on 3 Nov 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Highlights

Propose, then judge: a small calibrated decision model versus large language model judges for multilingual slot filling

Christophe Cruz, Hussam Ghanem, Hocine Cherifi

- A small fine-tuned judge beats 27B-180B LLM judges at choosing slot values
- LLM judges fail mainly by answering when the user expressed no value
- The small judge's confidences rank its errors far better: lower AURC
- A plain BIO tagger beats the pipeline when slot values are literal spans
- List augmentation helps only when it mimics real confusions between slots

Propose, then judge: a small calibrated decision model versus large language model judges for multilingual slot filling

Christophe Cruz[a,*], Hussam Ghanem[a] and Hocine Cherifi[a]

[a]*Laboratoire Interdisciplinaire Carnot de Bourgogne (ICB), UMR 6303 CNRS, Université Bourgogne Europe, 9 avenue Alain Savary, BP 47870, Dijon Cedex, 21078, France*

ARTICLE INFO

Keywords:
Small language models
Large language model judges
Calibration
Selective prediction
Slot filling
Multilingual dialogue systems
Abstention

ABSTRACT

Large language models are increasingly used as judges that pick one answer among options, but they are costly and their confidence rarely tells when a system may act on its own. We study this question in a hybrid architecture for task-oriented dialogue: inexpensive extractors propose candidate values for each slot, and a judge either selects one candidate or abstains. We compare Laya, a compact non-autoregressive decision model with fewer than half a billion parameters, used zero-shot and fine-tuned with a strictly proper scoring rule, against three open-weight large language model judges of 27 to 180 billion parameters on the same options, and against a fine-tuned sequence tagger. We cover three public benchmarks (MultiATIS++, MTOP and Multi3WOZ), English and French, single- and multi-turn inputs, and oracle and real candidate lists, with several seeds and Holm-corrected paired tests. With oracle candidates, the fine-tuned judge reaches a test slot F1 of 0.92–0.95, against 0.38–0.84 for the best large model, and all 72 paired comparisons of F1 and false positives favour it. Most of the gap comes from abstention: large models fill 15–21% of the slots that the user did not express on MultiATIS++, against 0.4–0.5% for the fine-tuned judge. Per request, it is 3 to 6 times faster, on a fraction of one GPU. After temperature scaling, its confidences rank its errors far better, with an area under the risk–coverage curve 3 to 40 times lower; at a 5% risk it decides 39–100% of the requests without asking, whereas no large model exceeds 1% on MTOP. The judge is, however, only as good as its candidates: when slot values are literal spans of the utterance, the tagger alone outperforms the whole pipeline with our proposers (F1 0.83–0.95 against 0.63–0.83), whereas the pipeline wins on multi-turn dialogue, where values come from the history or must be mapped to canonical forms. Augmenting the option lists during training makes the judge robust to longer lists and missing values only when it mimics realistic confusions between slots. A small calibrated judge is thus a practical decision component for hybrid systems, provided that its proposers are strong.

1. Introduction

Large language models (LLMs) are now routinely asked to act as *judges*: given a context and a short list of options, they return the option they find best. This pattern underlies LLM-as-a-judge evaluation (Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang, Lin, Li, Li, Xing, Zhang, Gonzalez and Stoica, 2023; Liu, Iter, Xu, Wang, Xu and Zhu, 2023), tool and function selection (Li, Chen, Ross, Huber, Moon, Lin, Dong, Sagar, Yan and Crook, 2024), and many agentic pipelines in which a model must decide whether to act, to do nothing, or to ask for clarification. Two properties matter as much as accuracy in such settings. The first is *cost*: a judge sits inside an interactive loop and is called many times per user turn. The second is *trustworthy confidence*: a system can act autonomously only if the judge's probability tells it reliably when it is likely to be wrong, so that uncertain cases can be deferred to the user or to a more expensive component (Geifman and El-Yaniv, 2017). LLM judges are known to be sensitive to the order and labels of options (Wang, Li, Chen, Cai, Zhu, Lin, Cao, Kong, Liu, Liu and Sui, 2024; Li, Chen, Ai, Chu, Zhou, Dong and Liu, 2025a), and their confidence must often be recalibrated before it can be used (Radharapu, Saxena, Li, Whitehouse, Williams and Cancedda, 2026).

This paper asks whether a small decision model, fine-tuned for the judging role and calibrated, can replace an open-weight LLM judge in such a loop. We study the question on a concrete and well-instrumented task: *slot filling* in task-oriented dialogue. For a request such as "a flight from *Lyon* to *Paris* on *Thursday morning*", the system must fill each slot of the current intent (departure city, arrival city, date, time…) with the value expressed by the user, or leave

*Corresponding author
christophe.cruz@ube.fr (C. Cruz); hussam.ghanem@ube.fr (H. Ghanem); hocine.cherifi@ube.fr (H. Cherifi)
ORCID(s): 0000-0002-5611-9479 (C. Cruz); 0009-0002-2652-5748 (H. Ghanem); 0000-0001-9124-4921 (H. Cherifi)

it empty. Recent work relies on generative LLMs for this task (Hu, Lee, Xie, Yu, Smith and Ostendorf, 2022; Heck, Lubis, Ruppik, Vukovic, Feng, Geishauser, Lin, van Niekerk and Gašić, 2023; Li et al., 2024). Generation is flexible, but it is costly, it can produce values that do not occur in the input, and the resulting scores are rarely usable to decide whether to act.

We instead separate two roles, following a long line of picklist-based dialogue state trackers (Lee, Lee and Kim, 2019; Zhang, Hashimoto, Wu, Wang, Yu, Socher and Xiong, 2020) and of retrieve-then-rerank pipelines (Wu, Petroni, Josifoski, Riedel and Zettlemoyer, 2020):

1. **Propose.** Inexpensive generators (a lexicon of training values, a zero-shot span extractor such as GLiNER (Zaratiana, Tomeh, Holat and Charnois, 2024), the ontology, the dialogue history) build, for each slot, a short list of candidate values from the current context.
2. **Judge.** A light judge either selects one candidate or answers "not specified". Its probability, once calibrated, drives a three-zone decision: *act* (fill the slot), *ignore* (leave it empty), or *ask* (defer).

Because the judge can only return one of the proposed candidates or abstain, it cannot hallucinate a value. Its errors therefore decompose cleanly into *candidate recall* (the right value was never proposed), *choice* (it was proposed but not selected) and *abstention* (a value was selected for a slot the user did not express), which makes the architecture convenient to diagnose.

As the judge we use Laya (Convai Innovations, 2026), a publicly released, non-autoregressive "System 1" decision model built on modern bidirectional encoders (Warner, Chaffin, Clavié, Weller, Hallström, Taghadouini, Gallagher, Biswas, Ladhak, Aarsen, Adams, Howard and Poli, 2025; Marone, Weller, Fleshman, Yang, Lawrie and Van Durme, 2026), which answers multiple-choice questions about a structured state in a single forward pass. We compare it, zero-shot and after fine-tuning, with three recent open-weight LLM judges of 27 to 180 billion parameters from two families (Qwen Team, 2026b,a; Gemma Team, 2026), all prompted with the same options and scored from the log-probabilities of their first answer token. The evaluation spans three public benchmarks with complementary properties: MultiATIS++ (Xu, Haider and Mansour, 2020) (one domain, single turn), MTOP (Li, Arora, Chen, Gupta, Gupta and Mehdad, 2021) (eleven domains and many intents, single turn) and Multi3WOZ (Hu, Zhou, Hergul, Gritta, Zhang, Iacobacci, Vulić and Korhonen, 2023) (multi-domain, multi-turn dialogue), in English and in French, with both oracle candidate lists (to measure the judge alone) and real ones (to measure the whole pipeline).

The general-AI question behind this setting is the trade-off between a large general model used zero-shot and a small model specialised for one decision. Recent studies show that small fine-tuned models can match or beat LLMs on classification and slot filling once a modest amount of labelled data is available (Pecher, Srba and Bielikova, 2025; Aguirre, Méndez, del Pozo, Torres and Torralbo, 2025; Rana, Hacioglu, Gopalan and Boothalingam, 2025). We add two dimensions that matter for deployment and that these studies do not measure: whether the judge *knows when it does not know* (risk–coverage behaviour after calibration), and how robust it is when the shape of its input changes (list length, missing gold value, language).

Research questions We organise the study around five questions: **RQ1** (efficiency) Does a fine-tuned Laya judge approach an open-weight LLM judge in slot F1, at a much lower cost? **RQ2** (decomposition) Which share of the error comes from candidate recall, and which from the choice, and how do they vary with the number of candidates k and the rate of missing gold values? **RQ3** (generalisation) Does the judge transfer from English to French? **RQ4** (abstention) Do calibration and the decision rule yield a useful risk–coverage trade-off? **RQ5** (training) Does random augmentation of the option lists at training time make the judge robust to a changing context?

Contributions

1. A **proposer–judge formulation** of slot filling as calibrated candidate selection with explicit abstention, and a reproducible conversion of three span-annotated benchmarks (two languages, single- and multi-turn) into this format, including a domain filter for multi-turn dialogue (Sections 3 and 4).
2. A **fine-tuning recipe** for a non-autoregressive decision model: supervised optimisation of the log score on training items produced by the inference code itself, so that training and evaluation cannot differ in format, with checkpoint selection on development F1 and an *option-list augmentation* designed to reproduce the confusions of real candidate generators (Section 3).
3. A **controlled comparison with three open-weight LLM judges** (27–180 B parameters, two families) on identical inputs: the fine-tuned judge is more accurate on every benchmark, language and candidate type, 3

to 6 times faster per request and 5 to 38 times faster in batch throughput, mainly because it abstains correctly (Section 5.1).

4. A **reliability analysis** showing that after calibration the small judge's confidences rank its errors far better than those of the LLMs (area under the risk–coverage curve 3 to 40 times lower), so that it can decide most requests without asking at a few percent risk, which none of the LLM judges can do on MTOP (Section 5.5).
5. A **comparison with a fine-tuned sequence tagger** that delimits where the proposer–judge architecture pays off: the tagger alone is better when slot values are literal spans of the utterance, the pipeline when values come from the dialogue history or must be mapped to canonical forms; the judge itself is not the bottleneck, the proposers are (Section 5.3).
6. An **error decomposition and robustness study** across candidate recall, choice and abstention, list size, missing values and cross-lingual transfer, with a negative-then-corrected result on augmentation that shows *which* augmentation helps (Sections 5.2, 5.4 and 5.6), all under a reproducibility protocol with multiple seeds, cluster bootstrap intervals, Holm-corrected paired permutation tests and code fingerprints in every run (Section 4.5).

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 formalises the proposer–judge architecture and describes the judges, the candidate generators, fine-tuning, augmentation and the calibrated decision. Section 4 details datasets, conditions, metrics and the statistical protocol. Section 5 answers RQ1–RQ5, Section 6 discusses errors, cost, calibration, limitations and threats to validity, and Section 7 concludes.

2. Related work

Our work sits at the crossing of five lines of research: natural language understanding (NLU) and dialogue state tracking, schema-guided and LLM-based approaches, LLMs as judges, candidate generation followed by selection, and calibration and selective prediction. A last paragraph positions it within the recent comparisons of small and large models.

Joint intent and slot models, and multilingual NLU Slot filling is classically cast as sequence labelling on top of a pre-trained encoder, jointly with intent classification (Chen, Zhuo and Wang, 2019). Multilingual benchmarks extend this setting to many languages: MultiATIS++ (Xu et al., 2020), distributed by the Linguistic Data Consortium as *ATIS – Seven Languages* (Mansour and Haider, 2021), MTOP (Li et al., 2021) and MASSIVE (FitzGerald, Hench, Peris, Mackie, Rottmann, Sanchez, Nash, Urbach, Kakarala, Singh, Ranganath, Crist, Britan, Leeuwis, Tur and Natarajan, 2023). These works provide our datasets and the cross-lingual protocols (train in English and test in another language, train in the target language, train on both). Sequence labellers are the natural proposers of our architecture and our extraction baseline (C0, Section 5.3).

Dialogue state tracking: choosing versus extracting Multi-domain dialogue state tracking has long opposed two strategies. Picklist approaches score every value of a fixed ontology, as in SUMBT (Lee et al., 2019); span or generation approaches extract or generate the value, as in TRADE (Wu, Madotto, Hosseini-Asl, Xiong, Socher and Fung, 2019); DS-DST combines both, picking categorical slots from a list and extracting non-categorical ones (Zhang et al., 2020). Our judge is a direct descendant of picklist tracking, with two differences: candidates are built *per request* from the current context instead of a fixed ontology, and the judge is a generic decision model that receives the slot description as text rather than a slot-specific classifier. Multilingual multi-turn data remain scarce; Multi3WOZ (Hu et al., 2023) and X-RiSAWOZ (Moradshahi, Shen, Bali, Choudhury, de Chalendar, Goel, Kim, Kodali, Kumaraguru, Semmar, Semnani, Seo, Seshadri, Shrivastava, Sun, Yadavalli, You, Xiong and Lam, 2023) provide culturally adapted dialogues, and we use the former for French multi-turn evaluation.

Schema-guided and LLM-based slot filling Describing slots in natural language enables zero-shot transfer to unseen services: the Schema-Guided Dialogue dataset (Rastogi, Zang, Sunkara, Gupta and Khaitan, 2020), slot descriptions for generative trackers (Lin, Liu, Moon, Crook, Zhou, Wang, Yu, Madotto, Cho and Subba, 2021) and description-driven modelling (Zhao, Gupta, Cao, Yu, Wang, Lee, Rastogi, Shafran and Wu, 2022); SGD-X measures robustness to paraphrased schemas (Lee, Gupta, Rastogi, Cao, Zhang and Wu, 2022). With LLMs, dialogue state tracking became an in-context learning (Hu et al., 2022), zero-shot prompting (Heck et al., 2023) or function-calling (Li et al., 2024) problem. Routing between small and large models reduces cost (Lee, Cheng and Ostendorf, 2024), and bounded neuro-symbolic agents validate state updates with a symbolic checker to avoid hallucinated values (Lin, Xiao,

Natarajan, Sankaranarayanan, Nawanit, Parashar, Zhang, Konaraddi, Mote and Niu, 2026). We share the motivation of predictability with the latter, but we enforce it by construction: the judge never generates a value. Our slot descriptions also follow the schema-guided tradition; robustness to renamed or paraphrased schemas in the spirit of SGD-X is left for future work (Section 6.6).

LLMs as judges and their biases LLM-as-a-judge evaluation correlates well with human preferences (Zheng et al., 2023; Liu et al., 2023), but LLM judges suffer from position bias and are not fair evaluators by default (Wang et al., 2024); permuting option positions or identifiers changes their decisions, which can be partly corrected by recalibrating the prediction distribution (Li et al., 2025a). Linear probes on hidden states give better calibrated judges than verbalised confidence (Radharapu et al., 2026). Our setting differs from evaluation: the judge acts *online*, inside the system, and its probability is used to decide whether to act. We use neutral letter keys, shuffle the options with a fixed seed for every judge, and read probabilities from first-token log-probabilities, which makes the LLM judge directly comparable to the decision model.

Generate, then select Entity linking pipelines retrieve candidates and then rerank them with a cross-encoder (Wu et al., 2020); zero-shot text classification can be cast as entailment between the input and a verbalised label (Yin, Hay and Roth, 2019); GLiNER extracts entity spans for arbitrary label descriptions with a bidirectional encoder (Zaratiana et al., 2024). Our architecture applies the same "generate cheaply, select carefully" principle to slot values, with GLiNER as one of the proposers.

Calibration and selective prediction Modern networks are often over-confident; temperature scaling (Guo, Pleiss, Sun and Weinberger, 2017) and isotonic regression (Zadrozny and Elkan, 2002) are simple post-hoc corrections. Selective classification lets a model abstain below a confidence threshold and characterises the trade-off by a risk–coverage curve (Geifman and El-Yaniv, 2017). Strictly proper scoring rules such as the log score reward honest probabilities (Gneiting and Raftery, 2007); Laya's original training uses rewards based on such rules (Convai Innovations, 2026), and we fine-tune it with the log score directly. Our three-zone decision (act, ignore, ask) is a selective-prediction policy whose threshold is set on development data for a target risk.

Small specialised models versus large general models Several recent studies compare fine-tuned small models with general LLMs. Around one hundred labelled examples suffice on average for a specialised small model to match a general LLM in text classification, although variance raises this number substantially (Pecher et al., 2025). In slot filling, a fine-tuned medium-size LLM outperforms zero-shot GPT-4o and JointBERT on MASSIVE (Aguirre et al., 2025), black-box distillation into a small model beats LLMs on industrial call-centre data (Rana et al., 2025), and small-model predictions can be injected into an LLM prompt to improve cross-domain zero-shot slot filling (Li, Huang, Huang, Xu and Fan, 2025b). Closest to our decision rule, Tikhomirov and Savchenko (2026) show that when an encoder forwards only its uncertain cases, a compact LLM verifier does not improve the cost–quality frontier, and that confidence gating on the encoder is preferable. Compared with these works, our judge is an order of magnitude smaller than the smallest LLMs considered (under half a billion parameters), the comparison is made on identical candidate lists with the same option format, and we evaluate the reliability of confidences (risk–coverage) and robustness to the shape of the input, in two languages and in multi-turn dialogue.

3. Problem formulation and method

3.1. Proposer–judge formulation

An *example* x is a user turn: the current utterance u, the preceding dialogue history h (empty for single-turn datasets), the current intent or domain ι, and, in multi-turn dialogue, the slots already filled before the turn, s^-. The schema gives, for each intent ι, the set $\mathcal{S}(\iota)$ of slots that it allows; each slot σ has a name and a short natural-language description (Section C). The gold annotation assigns to each slot either a set of values $V^*(x, \sigma)$ expressed in the turn, or nothing.

Figure 1 gives an overview of the architecture on a concrete request; this section formalises it. For every slot $\sigma \in \mathcal{S}(\iota)$ we issue one *request* $q = (x, \sigma)$. A set of proposers $G_1, \dots, G_m$ produces a candidate list

$$C(q) = \mathrm{dedup}\big(G_1(q) \cup \cdots \cup G_m(q)\big) = (c_1, \dots, c_k), \qquad k \le K, \tag{1}$$

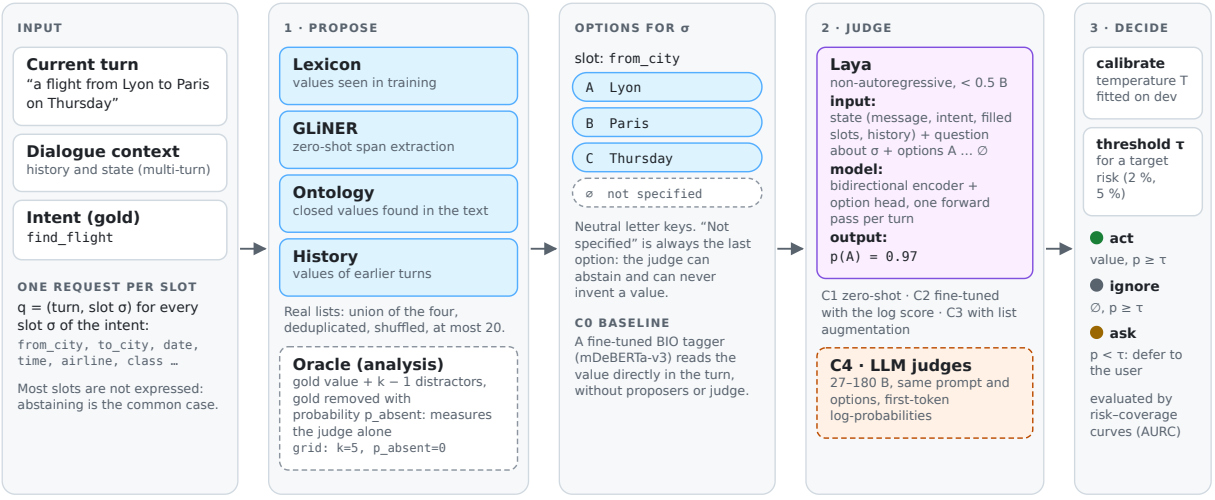


Figure 1: The proposer–judge architecture, illustrated on one request: the slot `from_city` of the turn “a flight from Lyon to Paris on Thursday”. **Input**: one request is issued for every slot of the gold intent, and most of these slots are not expressed. **1 Propose**: four inexpensive proposers (lexicon of training values, GLiNER, ontology, dialogue history) build the real candidate list; for analysis, an oracle builds a list around the gold value with $k-1$ distractors and removes the gold value with probability p_{absent}. **Options**: candidates appear under neutral letter keys, followed by “not specified” ($\perp$); the distractor C (*Thursday*) is the value of another slot of the same turn, the typical trap of real lists. **2 Judge**: Laya reads the dialogue state, the question about the slot and the options, and returns a probability per option in one forward pass (C1 zero-shot, C2 fine-tuned, C3 fine-tuned with option-list augmentation); in C4, an LLM judge is scored on the same options. **3 Decide**: after temperature scaling fitted on development data, a threshold τ set for a target risk maps the answer to *act* (fill the slot), *ignore* (leave it empty) or *ask* (defer to the user). C0, a fine-tuned BIO tagger, is the classical alternative that reads values directly in the turn, without proposers or judge.

where values are normalised before deduplication (Section C) and the first generator wins ties. A judge J maps the request and its list to a probability distribution over the k candidates and an extra option $\perp$ (“not specified”):

$$p(\cdot \mid q) = J\big(x, \sigma, C(q)\big) \in \Delta^{k}, \qquad \hat{y}(q) = \arg\max_{y \in C(q) \cup \{\perp\}} p(y \mid q), \qquad \hat{c}(q) = \max_{y} p(y \mid q). \tag{2}$$

The predicted value is $\hat{y}(q)$ unless $\hat{y}(q) = \perp$, in which case the slot stays empty. An empty list yields $\perp$ without calling the judge. The correct option is

$$y^{*}(q) = \begin{cases} c_i & \text{if } V^{*}(x,\sigma) \neq \emptyset \text{ and } c_i \in V^{*}(x,\sigma), \\ \perp & \text{otherwise (slot not expressed, or gold value absent from the list).} \end{cases} \tag{3}$$

Two properties follow directly. The judge cannot output a value that was not proposed, so it cannot hallucinate. And every error falls into exactly one of three classes, which we measure separately (Figure 2 and Section 4.4): a *recall* error (the gold value is not in $C(q)$), a *choice* error (it is in the list but another option is chosen, possibly $\perp$), or a *false positive* (a value is chosen for a slot the user did not express).

3.2. The Laya judge

Laya (Convai Innovations, 2026) is a non-autoregressive decision model released under the Apache-2.0 licence. It encodes a structured *state* (a dictionary of text fields) together with one or more *questions*; each question has typed options rendered as “`key: description`”, and a head attached to marker tokens returns a probability distribution over the options of each question in a single forward pass. We use the `multilingual` checkpoint, built on mmBERT-base (Marone et al., 2026) (the English checkpoint is built on ModernBERT-large (Warner et al., 2025)), with the runtime pinned to version 0.3.22 and the checkpoint to revision `7b928d8`. Its encoder accounts for 322 M parameters for the multilingual checkpoint (mmBERT-base; 421 M for the English one, according to the model cards), hence “fewer than half a billion” parameters in total.

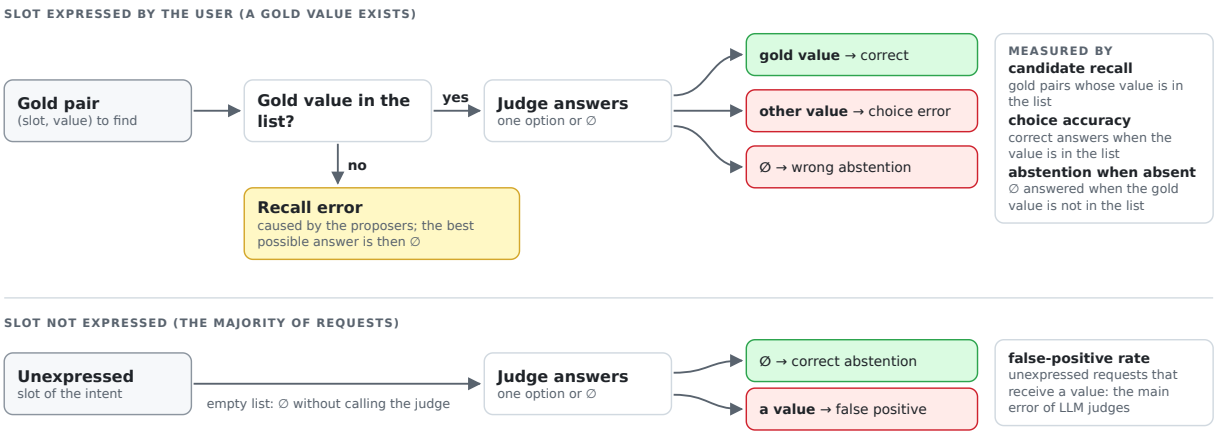


Figure 2: Error decomposition: each request ends in exactly one outcome. **Top**, a slot expressed by the user: either the gold value is missing from the list, a *recall* error of the proposers for which abstaining is the best possible answer, or it is present and the judge chooses it (correct), chooses another value (*choice* error) or abstains wrongly. **Bottom**, an unexpressed slot, the majority of requests: the judge must answer ⊥, and any value is a *false positive*; an empty list is answered ⊥ without calling the judge. The measures on the right separate the proposers (candidate recall) from the judge (choice accuracy, abstention when the gold value is absent, false-positive rate).

State The state sent to Laya contains, in this order (Laya truncates a state from the right): the last user message (at most 1,000 characters), the current intent (or domain), the slots already filled (multi-turn only) and the last four turns of history (at most 1,500 characters, truncated from the left).

Question In the main *single* mode, each request becomes one multiple-choice question whose instruction reads

```
Which value does the user's last message give for the slot "{slot}" ({description})?
```

with the candidates as options under neutral letter keys (A, B, …) and a last option "`not specified: the last message gives no value for this slot`". Candidates are shuffled with a seed that depends only on the run seed, the example and the slot, because Laya, like LLM judges (Wang et al., 2024), has a strong position bias. At most 15 candidates are shown during training and up to 20 during evaluation (Section A). A *two-step* mode, which first asks whether the slot is expressed and then which candidate is correct, is implemented (Section B), but its fine-tuned version was not evaluated in this study (hypothesis H4 remains open). All questions of the requests that share a state (same example, intent and filled slots) are grouped into one forward pass: one Laya call per turn.

3.3. Candidate generation

Oracle candidates To measure the judge alone, the oracle generator returns the gold value plus $k-1$ distractors, and removes the gold value with probability p_{absent} when the slot is expressed. Distractors are drawn by a sampler that mixes several strategies, each with a weight: values of the same slot seen in training (`train_values`), closed values of the ontology, values of *other* slots of the same utterance (`other_slots`), content n-grams of the utterance ($n \leq 3$), and values from the dialogue state or history. Within a strategy, a value is drawn with probability proportional to $\exp(\text{sim}/0.1)$, where sim is the character-trigram similarity with the gold value, which yields *hard* distractors. Our main preset, `hard_no_overlap`, additionally excludes boundary variants, i.e. values that contain the gold value or are contained in it as whole words ("on april" or "april 1" for "april"): choosing such a variant is a segmentation error rather than an understanding error, and with real candidates such variants appear naturally when a generator segments badly. The `hard` preset keeps them and serves as an ablation; the `easy` preset draws uniformly among all known values of all slots; the `confusion` preset (used for augmentation, Section 3.5) puts a weight of 3 on `other_slots`. Candidates are "blind" (no score, source or offsets), so that a judge cannot recognise the gold value from metadata. Three independent random streams, all derived from (seed, example, slot), decide the removal of the gold value, the sequence of distractors and the final order. Lists for k are therefore prefixes of lists for $k+1$, and examples without gold value are nested when p_{absent} grows, so that sweeps over k and p_{absent} compare paired conditions.

Real candidates To measure the whole pipeline, we use the union of four generators, in this priority order: (i) a *lexicon* of the normalised values seen in training for the slot, matched against the n-grams of the utterance; (ii) GLiNER (Zaratiana et al., 2024) (`urchade/gliner_multi-v2.1`), zero-shot, with the slot description as label and a threshold of 0.3; (iii) the *ontology*, whose closed values are matched exactly or approximately (similarity ≥ 0.85) in the utterance; and (iv) the *history* (Multi3WOZ only): the value already in the state, the same attribute in another domain, and known values mentioned in recent turns. The union keeps at most $K = 20$ candidates, deduplicated by normalised value, in a shuffled order. The fine-tuned judges are *not* retrained on real candidates: real lists are an out-of-distribution test of the judge trained on oracle lists.

3.4. Fine-tuning by the log score

Objective The released Laya package does not include its training module; the original model was trained with policy-gradient rewards derived from strictly proper scoring rules (Convai Innovations, 2026). We fine-tune it with direct supervision instead: the loss of a question with options $o_1, \ldots, o_n$ and correct option o_t is the log score $-\log p(o_t)$ (Gneiting and Raftery, 2007), i.e. the cross-entropy over its options. This is the same family of strictly proper rules, optimised by supervised learning rather than by reinforcement. We reimplemented the training loop on top of Laya's inference primitives (state encoding, collation and decision head), so that the encoding of a question at training time is exactly the one used for prediction, and we save checkpoints in Laya's own format.

Items built by the inference code Every training request goes through the function that prepares Laya's questions at evaluation time: same state, same instruction templates, same shuffled order, same truncation of the candidate list. Only the target is added: the position of the gold value among the options, or the "not specified" option when the slot is not expressed or the gold value is not in the list. A format mismatch between training and evaluation is therefore impossible by construction. Training requests are built on the training split with the evaluation's request builder (all slots allowed by the gold intent, Section 4).

Class imbalance Most requests concern slots that the user did not express (about 94% on MultiATIS++). We keep all requests with an expressed slot and at most two unexpressed requests per expressed request in each example (`unexpressed_ratio` $= 2$), drawn with the run seed; this gives 43,000 to 189,000 training questions per dataset and language (Table 14). On MultiATIS++ English this yields 43,007 training questions from 4,478 training utterances.

Optimisation and checkpoint selection We train for three epochs with AdamW (learning rates 2×10^{-5} for the encoder and 10^{-4} for the head), a linear warm-up over 5% of the steps followed by cosine decay, batches of 32 questions, gradient clipping at 1.0, bf16 autocast over fp32 weights and gradient checkpointing (Table 13). After each epoch we evaluate the slot F1 on all requests of the first 300 development examples (same distribution as at test time) and keep the best epoch. We first selected on the development loss, but the loss is distorted by over-confidence: one C3 seed kept an epoch whose loss was lower but whose decisions were worse; selecting on development F1 removed this failure mode (Section 6.7). Checkpoints are saved with neutral temperatures ($T = 1$); calibration is a separate, post-hoc step (Section 3.6).

Bilingual training For the joint English–French condition, the items of both languages are concatenated; each language keeps its own ontology (slot descriptions in that language where available) and its own oracle candidates fitted on its own training split. The checkpoint is selected on the English development set, with oracle candidates. The main configuration uses *monolingual* models (one per dataset and language); bilingual training and English-to-French transfer are analysed in Section 5.4, where bilingual training turns out to be unstable with real candidates.

3.5. Option-list augmentation

A judge trained on a single list shape (oracle, $k = 5$, gold always present) may fail when the context changes: longer lists, a missing gold value, other kinds of distractors. Our augmentation draws, for each training example, one combination of list size, missing-value rate and distractor preset, uniformly among the Cartesian product of the following lists (the order of options is already shuffled by the judge):

- **C3 v1** (generic): $k \in \{2, 5, 10, 20\}$, $p_{\text{absent}} \in \{0, 0.2, 0.5\}$, distractors $\in \{\texttt{easy}, \texttt{hard_no_overlap}\}$;
- **C3 v2** (confusion-oriented): $k \in \{2, 2, 3, 3, 5, 10, 20\}$ (short lists twice as likely), $p_{\text{absent}} \in \{0, 0.2, 0.5\}$, distractors $\in \{\texttt{confusion}, \texttt{hard_no_overlap}\}$.

Version 2 was designed after analysing the failure of version 1 on real candidates (Section 5.6): real lists are short and their main trap is a value of *another slot* of the same utterance ("boston" proposed as the stop-over city when it is the departure city), whereas the `easy` distractors of version 1 are of other types and easy to reject. On 200 training examples, 63% of the distractors for unexpressed slots come from other slots of the utterance with `confusion`, against 33% with `hard_no_overlap`. In the remainder of the paper "C3" denotes version 2; version 1 is kept as an ablation of the augmentation design. Note that with the judge's default of 15 options at training time, $k = 20$ is truncated to 15 (Section A).

3.6. Calibrated three-zone decision

Calibration The unit is a judged request: the judge's distribution over its options and the correct option y^*. Requests with an empty list are excluded, since their decision ($\perp$) is certain and identical for all judges. We fit on development data (i) *temperature scaling* (Guo et al., 2017), $p_T(y) \propto p(y)^{1/T}$ with T minimising the negative log-likelihood (NLL), which preserves the ranking of options and hence the decision, and (ii) *isotonic regression* (Zadrozny and Elkan, 2002) of the correctness of the selected option on its confidence. We report NLL, the multi-class Brier score and the expected calibration error (ECE, 15 equal-width bins).

Decision For a confidence $\hat{c}$ and a threshold τ, the policy is

$$\text{decision}(q) = \begin{cases} \text{ACT (fill } \hat{y}) & \text{if } \hat{c} \geq \tau \text{ and } \hat{y} \neq \perp, \\ \text{IGNORE (leave empty)} & \text{if } \hat{c} \geq \tau \text{ and } \hat{y} = \perp, \\ \text{ASK (defer to the user or a costlier judge)} & \text{if } \hat{c} < \tau. \end{cases} \tag{4}$$

Coverage is the share of requests decided without asking and *risk* the error rate among them (Geifman and El-Yaniv, 2017). Sweeping τ over all observed confidences (ties treated together) gives the risk–coverage curve, whose area (AURC; lower is better) summarises how well the confidences rank the errors. For a target risk (2% and 5% in the main grid), the threshold that maximises coverage is chosen on development data and then applied unchanged. As long as the test set is closed, we split the development set into two halves of clusters (dialogues or utterances), fit on one half, evaluate on the other, in both directions, and average (two folds). For the test-set results (Section 5.5), calibration and thresholds are fitted on the whole development set and applied once, unchanged, to the test set.

3.7. LLM judges

The LLM judges play exactly the same role. Each request is rendered with a fixed, versioned prompt (`slot-judge-v1`, reproduced in Section B) that contains the slot, its description, the intent, the filled slots, the last four turns of history, the last user message with its language, and the candidates under letter keys followed by a "not specified" option, in the same shuffled order as for Laya. The model is asked to answer with a single letter. We generate one token at temperature 0 and read the top-20 log-probabilities of that first token; the probabilities of all tokenisations of a letter ("A", "␣A"…) are summed and renormalised over the letters offered. If no letter appears in the top-20, the distribution is uniform and the case is logged (this never occurred in our checks, Section 4.3). Reasoning ("thinking") is switched off through the chat template, which we verified per model family. This logit-based reading gives the LLM a full probability distribution over the same options as Laya, which is required for a fair calibration comparison.

4. Experimental setup

4.1. Datasets and conversion

Figure 3 summarises the experimental design: the data, the two kinds of candidate lists, the compared conditions, the measures, and the separation between development and test data. The following subsections detail each part.

All three benchmarks annotate slots as text spans; we convert them to the candidate-selection format of Section 3.1 with a common schema and deterministic value normalisation (case, Unicode, numbers written in words, times, Section C). Each converted split is written with a manifest that records the SHA-256 of the raw and produced files. Table 1 summarises the data.

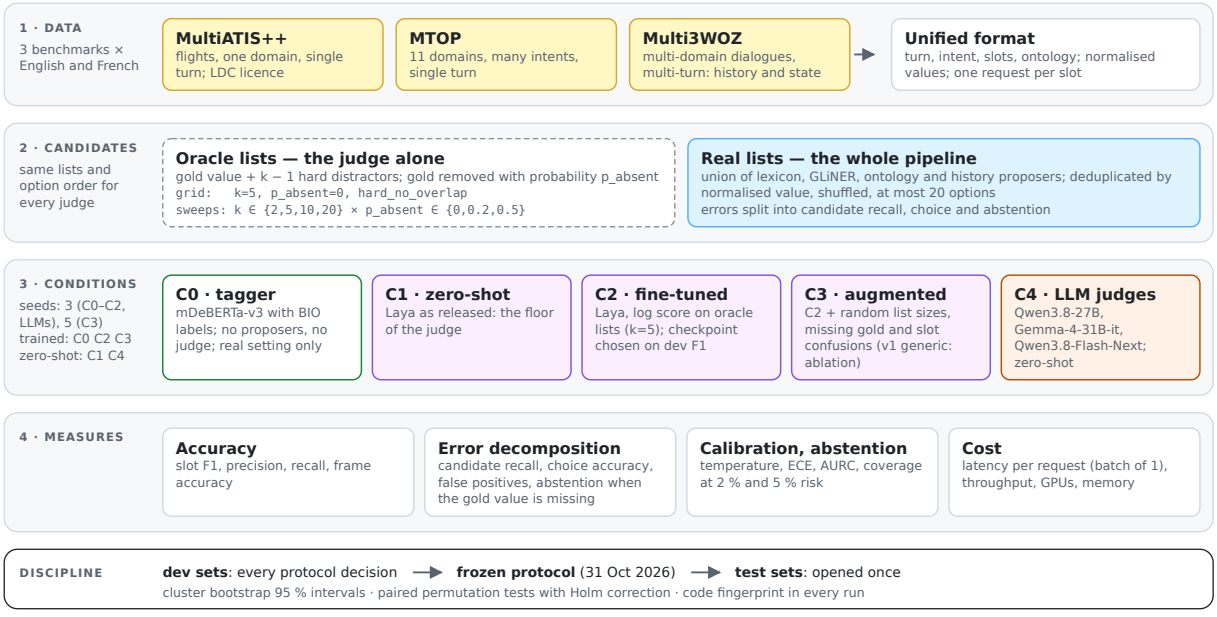


Figure 3: Experimental protocol. **(1)** Three span-annotated benchmarks, each in English and French, are converted to a unified format with one request per slot of the gold intent. **(2)** Every judge sees the same candidate lists in the same order: oracle lists isolate the judge (grid setting, and sweeps over the list size k and the missing-gold rate p_{absent}), real lists from four proposers measure the whole pipeline. **(3)** Five conditions: a fine-tuned BIO tagger (C0), Laya zero-shot (C1), fine-tuned (C2) and fine-tuned with option-list augmentation (C3), and three open-weight LLM judges (C4). **(4)** Four families of measures: accuracy, error decomposition, calibration and selective decision, and cost. All protocol decisions were taken on development sets; the protocol was then frozen and each test set opened once.

Table 1
Datasets after conversion. Split sizes in utterances (MultiATIS++, MTOP) or dialogues (Multi3WOZ, one example per user turn). Request volumes are the number of (example, slot) requests submitted to the judge on the development split.

Dataset	Lang.	Turns	Train / dev / test	Dev requests
MultiATIS++	EN, FR	single	4,478 / 500 / 893[a]	30,310 per language (1,682 EN / 1,678 FR gold pairs)
MTOP	EN	single	15,667 / 2,235 / 4,386	15,096
	FR	single	11,814 / 1,577 / 3,193	9,832
Multi3WOZ	EN, FR	multi	7,440 / 860 / 860 dial.	68,863 per language (7,708 gold pairs)

[a] The LDC release provides only train (4,978) and test (893); we draw the development set from the training set (the 500 identifiers with the smallest SHA-256), aligned across languages. This is not the 4,488/490 split of the original paper.

MultiATIS++ (Xu et al., 2020; Mansour and Haider, 2021): flight-information requests, one domain, with BIO labels. Spans are recovered as character offsets; for the rare multi-intent utterances (1–2%), the first intent is the main intent. Slot descriptions were written once by hand ("fromloc.city_name" → "departure city"), in English for English and in French for French, and frozen before evaluation. Thirteen lines of the French training split whose number of labels differs from the number of tokens are skipped.

MTOP (Li et al., 2021): eleven domains and many intents (113 intents and 75 slots in English, 110 intents and 73 slots in French). We keep top-level slots; a nested slot becomes one top-level slot whose value is the full covered text, and its sub-tree is kept for a separate analysis. Slot descriptions were written by hand in English and used for both languages.

Table 2
Compared conditions. All judges see the same candidate lists and the same option order for a given seed.

Code	Condition	Role
C0	Sequence tagger alone (mDeBERTa-v3-base, BIO labelling, fine-tuned)	classical extraction baseline (real setting only)
C1	Laya zero-shot (published `multilingual` checkpoint)	floor of the judge
C2	Laya fine-tuned, oracle lists $k = 5$, $p_{\text{absent}} = 0$	effect of fine-tuning
C3	Laya fine-tuned with confusion-oriented augmentation (v2)	robustness to list shape (RQ5)
C3 v1	Laya fine-tuned with generic augmentation	ablation of the augmentation design
C4	Open-weight LLM judges (three models), zero-shot	reference for the same role

Multi3WOZ (Hu et al., 2023): culturally adapted, multi-parallel versions of MultiWOZ dialogues, corrected release of 15 December 2023. One example is one user turn; its slots are the *difference* between the dialogue state after and before the turn, the definition consistent with joint goal accuracy. We keep the five usual domains (hotel, train, restaurant, attraction, taxi; 30 slots). The history and the state before the turn are given to the judges. Multi3WOZ has no per-turn intent; we therefore query only the slots of the gold domain of the turn (*domain filter*), which plays the role of the gold intent in the other datasets, and we pass the domain to the judge as a hint. Turns without a domain (15%) are queried on all 30 slots. On 300 development turns, the filter reduces the number of requests from about 30 to 11 per turn, of which 12.6% are expressed, and leaves out 8 of 420 gold pairs (1.9%) that belong to another domain, which caps recall. The alternative of querying all 30 slots at every turn would quadruple the requests, 99% of them unexpressed.

Requests In all datasets, the judge receives the gold intent (or domain) and is asked about every slot that this intent allows. This isolates slot filling from intent classification; end-to-end evaluation with predicted intents is left for future work. About 94% of the requests concern slots that are not expressed on MultiATIS++, which makes abstention a central part of the task.

4.2. Conditions

Table 2 lists the conditions. Each judge is evaluated with oracle candidates (`hard_no_overlap`, $k = 5$, $p_{\text{absent}} = 0$: the *grid setting*) and with real candidates. Sweeps over $k \in \{2, 5, 10, 20\}$ and $p_{\text{absent}} \in \{0, 0.2, 0.5\}$, the `hard` distractor preset and the English-to-French transfer are analyses, run for the Laya judges on MultiATIS++ English. Few-shot LLM judges, a generative LLM extractor without proposers, the two-step judge mode (Section B), unseen-domain splits and schema perturbations are left for future work (Section 6.6).

The sequence tagger (C0) C0 is the classical alternative to the proposer–judge architecture: a multilingual encoder, `microsoft/mdeberta-v3-base` (278 M parameters; revision `a048466`, weights from the safetensors conversion `refs/pr/5`), fine-tuned to label every word of the current utterance with a BIO tag of the slots of the ontology (`B-`σ for the first word of a value of slot σ, `I-`σ for the following words of that value, `O` outside any value; in "from New York to Paris", *New* is `B-from_city`, *York* is `I-from_city` and *Paris* is `B-to_city`), in the spirit of JointBERT (Chen et al., 2019) but without an intent head (the gold intent is given to all conditions). Words are those of the candidate generators, and the label of a word is read on its first sub-word. Training labels come from the gold spans of the current utterance only, so that values taken from the history, the dialogue context or canonical forms that do not occur in the text cannot be learnt. The model is trained for 10 epochs with AdamW (learning rate 3×10^{-5}, batch 32, linear warm-up over 10% of the steps), with three seeds, and the epoch with the best span F1 on the development set is kept. At inference, the decoded spans are normalised like the gold values; the best-scoring span of a slot is its only candidate, and C0 fills the slot with it, or answers "not specified" when the tagger proposes nothing. C0 therefore has no oracle setting.

Table 3
Judges. Revisions are Hugging Face commit hashes (abbreviated); all are recorded in `MODELES.lock.json`.

Judge	Architecture	Parameters	Precision	GPUs	Revision
Laya `multilingual` (Convai Innovations, 2026)	encoder, non-autoregressive	0.32 B (encoder)	fp32 (bf16 autocast)	1 (shared)	`7b928d8`
Qwen3.8-27B (Qwen Team, 2026b)	dense decoder	27.8 B	BF16	1	`1d4bf0f`
Gemma-4-31B-it (Gemma Team, 2026)	dense decoder	31.3 B	BF16	1	`842da37`
Qwen3.8-Flash-Next-FP8 (Qwen Team, 2026a)	MoE (10 of 512 experts active)	180 B	FP8	2	`236dfdf`

4.3. LLM judges and their configuration

We selected recent open-weight models that fit the hardware and cover two families and two architectures (Table 3). All are served offline by vLLM 0.30.0 in text-only mode, with a context of 4,096 tokens, greedy decoding of one token and reasoning disabled. On 20 development examples (940 requests), the three models never produced a first token without an option letter. A fourth model from a third family, Mistral-Small-4-119B, could not be loaded by vLLM 0.30.0 (its text configuration declares no architecture) and was withdrawn; it may be added with a later vLLM version. A preliminary model (Qwen3-32B) used during development is not reported in the main results. The LLM judges are served with vLLM (Kwon, Li, Zhuang, Sheng, Zheng, Yu, Gonzalez, Zhang and Stoica, 2023), version 0.30.0.

4.4. Metrics

Accuracy The main metric is the micro-averaged F1 over (slot, value) pairs, a "not specified" prediction not counting as a prediction, with precision and recall. *Frame accuracy* is the share of examples whose whole set of slots is exactly right, with no extra slot; it makes the cost of false positives visible. Joint goal accuracy, the official Multi3WOZ metric, is not reported: our requests are restricted to the gold domain and evaluated turn by turn, which does not match the cumulative dialogue state it scores.

Decomposition Because the F1 is dominated by abstention when most slots are unexpressed, we always report side by side: *candidate recall* (share of gold pairs whose value is in the list), *choice accuracy given present* (share of gold pairs correctly chosen when the value is in the list: the judge alone), the *false-positive rate* (share of unexpressed requests for which a value is chosen), and the *abstention rate when the gold value is absent* (share of requests with an expressed slot but no gold value in the list that are answered "not specified", the only answer that does not create an error).

Calibration and abstention NLL, Brier score, ECE, temperature T, AURC, and coverage at 1%, 2% and 5% risk (Section 3.6).

Cost Latency per request (p50 and p95), throughput (requests per second), number of GPUs and memory. Accuracy runs are batched (Laya batches of 128 states, LLM batches of 256 prompts), and their latencies are batch-amortised. Latency proper is measured in separate runs with batches of size one (one request per call to the judge), on the first 100 development examples of each English dataset with oracle lists in the grid setting, seed 13, with nothing else running on the GPUs. For Laya, the first occurrence of each input length costs about 80 ms, once (kernel selection for a new tensor shape); we report the p95 latency after this warm-up as well, since a deployed system would pad inputs to a few fixed lengths.

4.5. Statistical protocol

Seeds Fine-tuned conditions use three seeds (C0, C2) or five (C3); the seed controls the training order, the subsampling of unexpressed requests and the oracle candidate draws. LLM judges are deterministic; their seeds only change the candidate draws (three in the final grid, one in the current development runs).

Intervals and tests Confidence intervals (95%) are percentile bootstrap intervals with 1,000 resamples of *clusters* (dialogues for Multi3WOZ, utterances otherwise), never of isolated turns or requests. Each cluster is summarised by additive counts, and every metric is a ratio of sums with the same definitions as the point estimates. Paired comparisons use the clusters common to both systems: the difference and its interval come from a paired bootstrap, and the p-value from a permutation test that swaps the two systems cluster by cluster (10,000 permutations, two-sided). The Holm

correction is applied over all comparisons and metrics of a family (stated with each result). With 10,000 permutations the smallest attainable p-value is $1/10{,}001$, so the smallest Holm-corrected value depends on the family size; we report such floor values as "$p < 10^{-4}$ before correction".

Development and test discipline All protocol decisions (distractor preset, checkpoint criterion, augmentation version, monolingual versus bilingual training, decision thresholds) were taken on development data. The protocol of the final runs (grid, seeds, analyses, families of tests, decision procedure) was then written down and frozen on 31 October 2026, together with the fingerprint of the code (`docs/protocole_test.md` in the repository), and each test set was used in a single pass, from 31 October to 1 November 2026. During the early development phase, the MultiATIS++ English test split was opened several times; results obtained there are used only as exploratory evidence and are flagged as such (Section 6.7).

4.6. Hardware, software and reproducibility

All runs use one machine with two NVIDIA H200 NVL GPUs (143,771 MiB each, driver 595.71.05, CUDA 13.0 runtime from Python wheels). Laya fits on a fraction of one GPU (peak memory 1.7 GiB); the LLMs take one GPU (27–31 B) or both (180 B). The environment is locked with `uv` (Python 3.12.15, PyTorch 2.13.0, transformers 5.16.1, vLLM 0.30.0, laya 0.3.22, GLiNER 0.2.29); model revisions are pinned and runs are executed offline. Every run writes its resolved configuration, its predictions (one line per request, with the full probability vector) and its metrics, together with the hash of the configuration, the SHA-256 of the data manifests, and a fingerprint of all source and configuration files (`code_sha256`), which does not depend on the version-control metadata of the machine. Reruns with the same seed give identical metrics, and repeated trainings give the same losses to the third decimal. Section A lists all hyperparameters.

5. Results

All results in this section are **test-set results** of the frozen protocol (Section 4.5), obtained in a single pass after every decision had been taken on development data. Unless stated otherwise, values are means over seeds (three for C0, C1, C2 and the LLM judges, five for C3) with the standard deviation across seeds; seed-to-seed standard deviations of the slot F1 are at most 0.012 for every judge in the main grid. The main configuration uses monolingual models (one per dataset and language); bilingual training is analysed in Section 5.4. Paired comparisons use seed 13 against seed 13, cluster permutation tests and the Holm correction within the three families of the protocol: C2 against each LLM judge (108 tests), C3 against C2 (36 tests), and C0 against the best Laya judge chosen on development data (18 tests). Latencies were measured on development examples (Section 4.4). The per-run metrics, intervals and comparison tables are in the code repository.

5.1. RQ1: accuracy and cost against LLM judges

A small fine-tuned judge is far more accurate Table 4 compares the judges on MultiATIS++ English. Zero-shot, Laya is not a usable slot judge: its F1 is 0.025 in the grid setting and 0.275 with real candidates, it fills 83% of the unexpressed slots with oracle lists, and its choice accuracy given a present gold value is only 0.20: the information is missing from the model, not from the decision rule. After fine-tuning, it reaches an F1 of 0.941 with oracle candidates and 0.828 with real ones. The three LLM judges stay between 0.36 and 0.43 (oracle) and 0.52 and 0.53 (real).

The gap is an abstention gap on MultiATIS++ On this dataset, the LLM judges choose the right value at least as well as the fine-tuned judge when it is in the list (choice accuracy 0.87–0.97, against 0.87–0.96), including the 180 B mixture-of-experts model: in the 12 paired comparisons of choice accuracy on MultiATIS++ (two languages, two candidate types, three LLMs), the differences range from −0.022 to 0.000, five of them significant in favour of the LLM. The whole difference comes from abstention: the LLM judges fill 15–21% of the unexpressed slots with oracle lists and 7% with real lists, against 0.4–0.5% and 0.4–0.7% for the fine-tuned judges (C2 and C3). Frame accuracy, which requires all slots of an utterance to be right and no extra one, makes this visible: 0.79 for C2 against at most 0.04 for the LLMs. Size does not solve the problem: the 180 B model is no better than the 27–31 B dense models.

The result holds across datasets, languages and families Table 5 extends the comparison to the three benchmarks in English and French. A fine-tuned judge is the best *judge* in every row (the sequence tagger C0 is discussed in Section 5.3). Of the 108 paired comparisons between C2 and the three LLMs (6 dataset–language pairs × 2 candidate

Table 4
MultiATIS++ English, test set: fine-tuned Laya against three open-weight LLM judges. Oracle: `hard_no_overlap`, $k = 5$, $p_{\text{absent}} = 0$. Mean ± s.d. over seeds (C2 and LLMs: 3; C3: 5). "Choice" = choice accuracy given that the gold value is in the list; FP = false-positive rate on unexpressed slots; frame acc. = share of utterances whose whole set of slots is exactly right.

Cand.	Judge	F1	P	R	Frame acc.	Choice	FP
oracle	C2 Laya FT	**0.941** ± 0.002	**0.932**	0.951	**0.785**	0.956	**0.004**
	C3 Laya FT + aug.	0.935 ± 0.004	0.927	0.942	0.759	0.947	**0.004**
	Gemma-4-31B	0.427 ± 0.001	0.275	0.955	0.038	0.959	0.145
	Qwen3.8-Flash-Next (180 B)	0.395 ± 0.001	0.248	**0.968**	0.025	**0.973**	0.169
	Qwen3.8-27B	0.361 ± 0.001	0.222	0.964	0.025	0.968	0.194
real	C2 Laya FT	0.828 ± 0.002	**0.829**	0.826	**0.549**	0.866	**0.004**
	C3 Laya FT + aug.	**0.829** ± 0.007	**0.829**	0.830	0.542	0.870	0.005
	Gemma-4-31B	0.531 ± 0.002	0.391	0.828	0.084	0.868	0.069
	Qwen3.8-Flash-Next (180 B)	0.528 ± 0.003	0.384	**0.843**	0.085	**0.884**	0.072
	Qwen3.8-27B	0.522 ± 0.003	0.379	0.838	0.084	0.879	0.073

Table 5
Slot F1 on the test sets of the three benchmarks (mean over seeds). "Recall" is candidate recall of the real lists (1.000 for oracle lists by construction). C0: sequence tagger alone (real setting only; in italics when it beats every judge); C1: zero-shot Laya; C2: fine-tuned Laya; C3: fine-tuned with confusion-oriented augmentation; three seeds except C3 (five). Best judge per row in bold.

Dataset	Lang.	Cand.	Recall	C0	C1	C2	C3	Qwen3.8-27B	Gemma-4-31B	Flash-Next 180 B
MultiATIS++	EN	oracle	1.000	–	0.025	**0.941**	0.935	0.361	0.427	0.395
	EN	real	0.959	*0.949*	0.275	0.828	**0.829**	0.522	0.531	0.528
	FR	oracle	1.000	–	0.025	**0.925**	0.921	0.339	0.383	0.368
	FR	real	0.948	*0.923*	0.276	**0.832**	0.828	0.525	0.539	0.556
MTOP	EN	oracle	1.000	–	0.143	**0.953**	0.951	0.704	0.753	0.727
	EN	real	0.751	*0.904*	0.433	0.690	**0.695**	0.563	0.583	0.558
	FR	oracle	1.000	–	0.148	**0.951**	0.942	0.706	0.753	0.736
	FR	real	0.675	*0.833*	0.335	0.625	**0.629**	0.525	0.529	0.528
Multi3WOZ	EN	oracle	1.000	–	0.072	0.940	**0.946**	0.822	0.844	0.813
	EN	real	0.888	0.733	0.196	**0.853**	0.852	0.750	0.762	0.744
	FR	oracle	1.000	–	0.056	0.917	**0.919**	0.752	0.753	0.730
	FR	real	0.550	0.380	0.121	**0.592**	0.588	0.459	0.474	0.458

types × 3 LLMs × 3 metrics, Holm), 96 are significant in favour of C2: all 36 on slot F1, all 36 on the false-positive rate, and the 24 on choice accuracy outside MultiATIS++. The F1 gains are +0.51 to +0.59 on MultiATIS++ with oracle lists and +0.28 to +0.32 with real lists, +0.20 to +0.25 and +0.09 to +0.13 on MTOP, and +0.10 to +0.19 and +0.10 to +0.14 on Multi3WOZ. Unlike MultiATIS++, the LLMs also choose less well on MTOP and Multi3WOZ (choice accuracy 0.73–0.89 against 0.83–0.96 for C2; differences +0.06 to +0.18): MTOP has many intents and slots, and Multi3WOZ requires reading the history and database-like values. Multi3WOZ is the most favourable dataset for the LLMs (false-positive rates of 1.6–2.6%, thanks to the domain filter), MTOP and MultiATIS++ the least favourable (15–21% with oracle lists). Among the LLMs, Gemma-4-31B is the best in 11 of the 12 settings.

Cost Table 6 gives the latency of one request judged alone, measured on development examples. The fine-tuned judge answers in 7.9–8.5 ms (p50) on a fraction of one GPU, against 24.6–35.1 ms for Gemma-4-31B, 34.7–50.5 ms for Qwen3.8-27B and 40.5–49.4 ms for the 180 B model on two GPUs: a ratio of 3.1 to 6.0. Once each input length has been seen, its p95 latency stays within 1 ms of the median (8.0–9.4 ms); the first occurrence of a length costs about 80 ms once, which dominates the raw p95 on MTOP and Multi3WOZ, whose 100 examples contain many distinct lengths. With batches, the gap widens: with oracle lists on MultiATIS++, the fine-tuned judge processes 691 requests per second on one GPU, against 83 for the 180 B model on two GPUs and 38 and 18 for the dense 31 B and 27 B

Table 6
Latency of one request judged alone (batch size 1), first 100 development examples of each English dataset, oracle lists in the grid setting, seed 13. Each cell: p50 / p95 after warm-up (ms), i.e. excluding the first occurrence of each input length, which costs Laya about 80 ms once. GPUs: number of H200 used.

Judge	GPUs	MultiATIS++	MTOP	Multi3WOZ
C2 Laya FT	1 (shared)	**7.9** / 8.0	**8.0** / 8.4	**8.4** / 9.4
C3 Laya FT + aug.	1 (shared)	8.0 / 8.1	8.1 / 8.5	8.5 / 9.4
Gemma-4-31B	1	24.6 / 25.7	24.6 / 25.8	35.1 / 47.9
Qwen3.8-27B	1	34.8 / 38.3	34.7 / 35.7	50.5 / 59.3
Qwen3.8-Flash-Next (180 B)	2	40.7 / 42.5	40.5 / 41.9	49.4 / 53.9

models on one GPU (a throughput ratio of 8 to 38); on Multi3WOZ, where the history makes inputs longer, 360–370 requests per second against 28–74 (ratio 5 to 13). Laya's peak GPU memory is below 2 GiB, against 55.6 to 185.5 GB of weights alone for the LLMs. Fine-tuning costs about 80 s per epoch on one H200 for MultiATIS++ (4 min for three epochs) and about 1 h per seed for the largest training set, bilingual Multi3WOZ (377,033 questions). The tagger C0 trains in 1 to 9 min and labels a whole utterance, all slots at once, in 6.5–17.7 ms, whereas the judge needs one pass per non-empty candidate list (about 5 per utterance on MultiATIS++ with real candidates). The whole study used about 120 H200 GPU-hours (100 h for the 1,215 evaluation runs, of which 52 h for the LLM judges, and 20 h for the 102 fine-tunings), a lower bound since runs discarded early in the study are not counted.

Hypothesis H1 H1 stated that with oracle candidates the fine-tuned judge would stay within 3 F1 points of the LLM judge, at least ten times faster. The accuracy part is exceeded by a wide margin, since the small judge is better, not merely close. The latency part is *not* supported as stated: request by request, the ratio is 3 to 6, not 10. A ratio of 10 or more is reached only in batch throughput (8 to 38 on MultiATIS++), and the small judge needs a fraction of one GPU where the LLMs need one or two.

5.2. RQ2: where does the error come from?

With real candidates, recall is the first source of error The gap between oracle and real F1 for the fine-tuned judges (Table 5) closely follows candidate recall. The real lists contain the gold value for 75% and 68% of the gold pairs on MTOP (English, French), 89% and 55% on Multi3WOZ, and 96% and 95% on MultiATIS++, while the choice accuracy of C2 given that the value is present remains 0.83–0.91. Hypothesis H2 is therefore supported on all datasets. The proposers are markedly weaker in French: on Multi3WOZ French the real F1 is capped at 0.59 for an oracle F1 of 0.92. The judge itself works well in French (Section 5.4); improving the candidates is the most profitable direction, as the comparison with a sequence tagger shows (Section 5.3).

Real lists are short, and their trap is confusion between slots On MultiATIS++ English (development set), the real lists contain on average 0.27 candidates per request; 89% of the unexpressed requests have an empty list and are answered "not specified" without calling the judge, and the non-empty lists have 1.7 candidates on average, mostly values of *other* slots of the utterance. Choice is therefore harder than with oracle lists (choice accuracy 0.87 against 0.96 on the test set, Table 4), because the competing candidates come from the same utterance.

With oracle lists, choice is robust and abstention decides When p_{absent} removes the gold value, candidate recall drops for all judges alike, the F1 is capped, and the judges are separated by precision and by their ability to abstain when the gold value is missing (Table 12). With the gold value present, the choice accuracy of the fine-tuned judges stays above 0.90 up to $k = 20$, whereas zero-shot Laya falls below 0.10. The sweeps were run for the Laya judges on MultiATIS++ English only.

Boundary variants With the `hard` preset, which allows distractors that contain or are contained in the gold value, the choice accuracy of the zero-shot judge halves (from 0.203 to 0.094). A C2 judge trained on the same preset is unaffected (choice accuracy 0.954 against 0.956, F1 0.936 against 0.941, three seeds), whereas C3, whose augmentation never shows boundary variants, drops to 0.841 (F1 0.864). Part of what looks like a "choice" error is a segmentation error, which a judge learns to avoid only if it is trained on it. The LLM judges were not evaluated on this preset.

Table 7
Why the tagger wins on single-turn datasets and loses on Multi3WOZ: a tagger can only return a span of the current turn. Illustrative examples (the second one is a Multi3WOZ development turn).

Situation	Turn (and context)	Expected value	C0	Pipeline
The value is a span of the turn	"a flight from *Boston* to Denver on Monday"	`boston`	yes	yes
The value comes from an earlier turn	earlier: "... the restaurant *Sitar Tandoori*"; now: "I also need a taxi to *the restaurant*"	`sitar tandoori` (taxi destination)	no	yes (history proposer)
The value is a canonical form	"un restaurant *cher*"; "il me faut *cinq tickets*"	`expensive`; `5`	no	yes (ontology proposer)

Table 8
Sequence tagger alone (C0, 3 seeds) against the proposer–judge pipeline with real candidates (best of C2 and C3 on the test set, Table 5), test sets. "Recall": share of gold pairs whose value is the tagger's best span (C0) or is in the real list (pipeline).

Dataset	Lang.	F1 C0	P C0	R C0	Recall C0	Recall lists	F1 pipeline
MultiATIS++	EN	**0.949** ± 0.002	0.962	0.937	0.958	0.959	0.829
	FR	**0.923** ± 0.005	0.935	0.911	0.931	0.948	0.832
MTOP	EN	**0.904** ± 0.002	0.918	0.891	0.917	0.751	0.695
	FR	**0.833** ± 0.003	0.849	0.817	0.838	0.675	0.629
Multi3WOZ	EN	0.733 ± 0.002	0.832	0.655	0.666	0.888	**0.853**
	FR	0.380 ± 0.002	0.429	0.340	0.346	0.550	**0.592**

5.3. Against a sequence tagger

Table 8 compares the proposer–judge pipeline with the classical alternative: a fine-tuned BIO tagger that reads values directly in the utterance (C0, Section 4.2). The tagger is *better* than the whole pipeline on the two single-turn datasets, by +0.09 to +0.22 F1 in both languages (family 3, Holm, all significant). Its precision is high (0.85–0.96) and its false-positive rate stays low (0.2–1.4%), so it does not suffer from the abstention problem of the LLM judges. The reason is recall: its single best span contains the gold value for 84–96% of the gold pairs, against 68–75% for the union of our four generic proposers on MTOP. The comparison is reversed on Multi3WOZ, where the pipeline wins by 0.12 (English) and 0.21 (French), also significantly. There, 20% of the gold pairs of the development set have no span in the current utterance (references to the history, values of the dialogue context), and among those that do, the gold value differs from the normalised surface form in 9% of the cases in English and 54% in French, where gold values are canonical values ("prix cher" → *expensive*, "cinq tickets" → *5*). In all, 27% (English) and 63% (French) of the gold values are not literal spans of the utterance. A tagger cannot produce these values; the ontology and history proposers can, and the judge selects them reliably. Table 7 illustrates the three situations. (Choice accuracy is not informative for C0, whose single candidate is chosen by construction.)

Where the pipeline loses, the judge is not the cause On MultiATIS++, the real lists contain the gold value as often as the tagger finds it (0.959 against 0.958 in English), yet the pipeline loses 0.12 F1: the lists also contain values of other slots of the utterance, and the judge, trained on oracle lists, confuses them (choice accuracy 0.87, Section 5.2). On MTOP, the gap comes first from candidate recall. In both cases the fix lies on the proposer side: the tagger's spans are the obvious candidates to add, and the judge would then decide among the tagger's proposal, the lexicon, the ontology and the history. We have not run this combination, which would change the real-candidate setting of the frozen grid; it is the most direct next step.

5.4. RQ3: cross-lingual transfer

Table 9 evaluates on the French test sets judges trained on French only (FR→FR), on English only (EN→FR, no French data, checkpoint selected on the English development set) and on both (EN+FR→FR).

Table 9
Cross-lingual transfer: slot F1 on the French test sets (mean ± s.d. over seeds: 3 for C2 and 5 for C3; bilingual models: the existing checkpoints, two seeds for C2 and three for C3 on Multi3WOZ). †Unstable across seeds: F1 of seeds 13, 14 and 15.

Dataset	Cand.	Judge	FR→FR	EN→FR (no FR data)	EN+FR→FR
MultiATIS++	oracle	C2	0.925 ± 0.003	0.833 ± 0.015	**0.939** ± 0.001
	oracle	C3	0.921 ± 0.004	0.757 ± 0.047	**0.930** ± 0.004
	real	C2	0.832 ± 0.007	0.778 ± 0.015	**0.846** ± 0.005
	real	C3	0.828 ± 0.009	0.775 ± 0.014	**0.829** ± 0.010
MTOP	oracle	C2	0.951 ± 0.002	0.910 ± 0.002	**0.958** ± 0.002
	oracle	C3	0.942 ± 0.002	0.896 ± 0.004	**0.952** ± 0.002
	real	C2	**0.625** ± 0.003	0.610 ± 0.003	0.616 ± 0.003
	real	C3	**0.629** ± 0.006	0.617 ± 0.010	**0.629** ± 0.012
Multi3WOZ	oracle	C2	**0.917** ± 0.003	0.833 ± 0.003	0.916 ± 0.002
	oracle	C3	0.919 ± 0.001	0.801 ± 0.012	**0.923** ± 0.002
	real	C2	**0.592** ± 0.007	0.554 ± 0.010	0.565 ± 0.007
	real	C3	**0.588** ± 0.012	0.555 ± 0.006	0.506 / 0.578 / 0.571†

The judge works in French; zero-shot transfer costs 0.04–0.09 F1 Trained and tested in French, the judge loses little with respect to English with oracle lists (−0.002 to −0.023 F1). Without any French data, C2 loses 0.092 on MultiATIS++, 0.041 on MTOP and 0.084 on Multi3WOZ with oracle lists: the multilingual checkpoint transfers partly, without matching in-language training. With real lists the transfer loss is smaller (−0.015 to −0.054), because the ceiling is then set by the French proposers rather than by the judge. The augmented judge transfers worse without French data (0.757 ± 0.047 against 0.833 on MultiATIS++, with a large variance across seeds): an augmentation tuned on English distractors does not help the change of language.

Bilingual training: stable with oracle lists, unstable with real ones With oracle lists, bilingual training matches or exceeds French-only training (−0.001 to +0.014) and costs nothing in English (EN→EN versus EN+FR→EN test F1: MultiATIS++ C2 0.941 → 0.947, C3 0.935 → 0.946; MTOP C2 0.953 → 0.959; Multi3WOZ C2 0.940 → 0.943). With real candidates, however, it does not transfer this benefit: on the French test sets, bilingual minus monolingual F1 is +0.014, −0.009 and −0.027 for C2 and +0.001, 0.000 and −0.036 for C3 (MultiATIS++, MTOP, Multi3WOZ). On Multi3WOZ, the bilingual C3 is unstable across seeds with real lists: F1 0.714, 0.845 and 0.802 in English (against 0.852 ± 0.006 for the monolingual model) and 0.506, 0.578 and 0.571 in French (against 0.588). The cause is an excess of wrong abstentions: the bilingual checkpoints refuse 644 to 1,979 gold values that are present in the list, against 398 to 800 for the monolingual ones, whereas the same checkpoints are normal with oracle lists. This instability had been observed on the development sets, where it motivated the choice of monolingual models for the main configuration; the test set confirms it. A likely explanation is checkpoint selection: the epoch is chosen on the *English* development set with *oracle* candidates, which says nothing about abstention on real French lists. Selecting the checkpoint on a development set that includes real candidates and both languages is a direct way to make bilingual training reliable.

5.5. RQ4: calibration, abstention and risk–coverage

All judges are over-confident, the LLMs much more so Temperature scaling fitted on development data always gives $T > 1$, i.e. probabilities must be flattened. With real lists, T ranges from 1.8 to 4.5 for the fine-tuned judges (largest on MTOP, where real lists are furthest from the training lists) and from 1.6 to 16.3 for the LLM judges; Gemma-4-31B, the most accurate LLM, is also the most over-confident (T = 3.7–16.3). Scaling, fitted on the development set and applied unchanged to the test set, divides the ECE of the fine-tuned judges by 2 to 8 (e.g. 0.063 → 0.016 for C2 with real lists on MultiATIS++ English).

The small judge's confidences rank its errors far better Calibration changes the confidence scale but not the ranking of requests; what matters for a selective policy is whether high confidences coincide with correct decisions, which AURC measures. Table 10 reports, for real candidates, the AURC and the coverage obtained on the test set when the threshold is chosen on the development set for a 5% risk. The best fine-tuned judge has an AURC 3 to 40 times lower than every LLM judge in all six dataset–language pairs. On MultiATIS++ and MTOP, no LLM decides

Table 10
Selective decision with real candidates, test sets (seed 13). Temperature, isotonic map and thresholds are fitted on the development set and applied unchanged. Each cell: AURC (lower is better) · coverage on the test set at the 5% target risk (share of requests decided without asking). Best AURC per row in bold.

Dataset, lang.	C2	C3	Qwen3.8-27B	Gemma-4-31B	Flash-Next 180 B
MultiATIS++ EN	0.029 · 1.00	**0.015** · 1.00	0.370 · 0.00	0.444 · 0.00	0.520 · 0.00
MultiATIS++ FR	**0.013** · 1.00	0.015 · 0.95	0.283 · 0.00	0.365 · 0.00	0.399 · 0.00
MTOP EN	0.071 · 0.39	**0.062** · 0.50	0.217 · 0.00	0.212 · 0.00	0.239 · 0.00
MTOP FR	0.047 · 0.68	**0.044** · 0.66	0.158 · 0.00	0.194 · 0.00	0.198 · 0.01
Multi3WOZ EN	**0.003** · 1.00	**0.003** · 1.00	0.009 · 1.00	0.028 · 1.00	0.028 · 1.00
Multi3WOZ FR	**0.003** · 1.00	0.004 · 1.00	0.027 · 0.94	0.060 · 0.00	0.109 · 0.82

Table 11
Calibration and selective decision on MultiATIS++ English, real candidates, test set (seed 13; calibration and thresholds fitted on the development set). T: fitted temperature; ECE before → after temperature scaling; coverage and realised risk at the 1% and 5% target risks.

Judge	T	ECE	AURC	Cov. @1%	Cov. / risk @5%
C2	3.17	0.063 → 0.016	0.029	0 (unreachable)	0.997 / 0.068
C3	2.73	0.065 → 0.025	**0.015**	**0.792**	0.999 / 0.070
LLM judges (3 models)	5.1–16.3	0.37–0.45 → 0.09–0.16	0.37–0.52	0	0

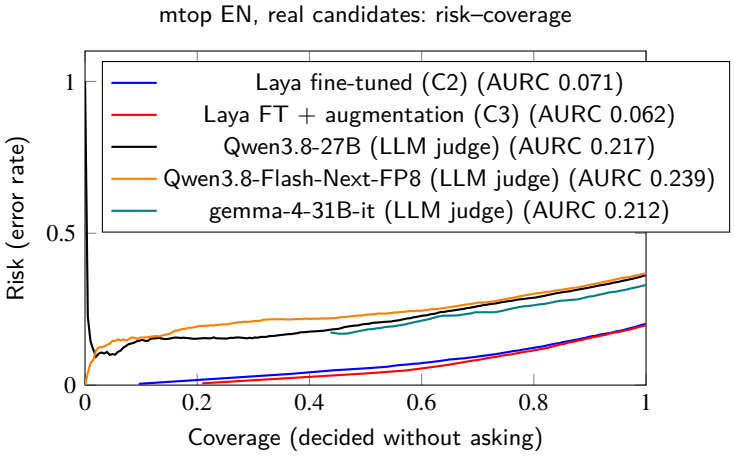


Figure 4: Risk–coverage curves with real candidates, MTOP English test set, seed 13 (one curve per judge; the legend gives the AURC). The curves for the other dataset–language pairs are generated in the same folder of the repository.

more than 0.5% of the requests on its own at the 5% target, whereas the fine-tuned judges decide 39–68% of the MTOP requests and 95–100% of the MultiATIS++ requests. On Multi3WOZ, most judges reach 82–100% coverage at the 5% target (Gemma-4-31B in French: none), but only the fine-tuned judges keep it at 2% (99–100%, against 0–88% for the LLMs). Thresholds tuned on development data transfer imperfectly: the risk realised on the test set exceeds the 5% target on MultiATIS++ (6.2–7.0% for the fine-tuned judges, Table 11) and stays at or below about 5% on MTOP and Multi3WOZ.

Augmentation helps calibration on MultiATIS++, less elsewhere At equal F1 on real lists (Table 4), C3 ranks its certainties twice as well as C2 on MultiATIS++ English (AURC 0.015 against 0.029): at a 1% target C2 can decide nothing without asking, whereas C3 decides 79% of the requests. This advantage does not generalise: C3 is slightly better on MTOP and C2 slightly better on MultiATIS++ French and Multi3WOZ (Table 10).

Table 12
Robustness to list size and missing gold value, MultiATIS++ English test set: C2 (trained on $k = 5$, $p_{\text{absent}} = 0$; 3 seeds) against C3 (confusion-oriented augmentation; 5 seeds). Abst. = correct abstention rate when the gold value is missing from the list (undefined for $p_{\text{absent}} = 0$).

Setting	F1 C2	F1 C3	P C2	P C3	Abst. C2	Abst. C3
Real candidates	0.828 ± 0.002	**0.829** ± 0.007	0.829	0.829	0.88	0.88
$k = 2$, $p_{\text{absent}} = 0$	**0.952** ± 0.001	0.948 ± 0.003	0.953	0.950	–	–
$k = 5$, $p_{\text{absent}} = 0$	**0.941** ± 0.002	0.935 ± 0.004	0.932	0.927	–	–
$k = 5$, $p_{\text{absent}} = 0.5$	0.570 ± 0.017	**0.600** ± 0.010	0.735	0.849	0.79	**0.98**
$k = 10$, $p_{\text{absent}} = 0$	0.916 ± 0.008	**0.928** ± 0.004	0.885	0.926	–	–
$k = 10$, $p_{\text{absent}} = 0.5$	0.533 ± 0.025	**0.595** ± 0.012	0.624	0.851	0.67	**0.98**
$k = 20$, $p_{\text{absent}} = 0$	0.828 ± 0.038	**0.925** ± 0.005	0.749	0.938	–	–
$k = 20$, $p_{\text{absent}} = 0.5$	0.463 ± 0.044	**0.593** ± 0.011	0.473	0.872	0.57	**0.98**

5.6. RQ5: robustness through option-list augmentation

Generic augmentation: robust on oracle sweeps, harmful on real lists We first trained C3 v1, whose augmentation mixes easy random distractors and hard ones (five seeds, same training recipe as the other judges). It behaves as intended on oracle sweeps: outside the training setting of C2 ($k = 5$, gold always present), C2 degrades (F1 from 0.952 at $k = 2$ to 0.828 at $k = 20$; correct abstention when the gold value is missing from 0.79 at $k = 5$ to 0.57 at $k = 20$), while C3 v1 remains stable (abstention 0.97–0.98 at every k; F1 0.918 at $k = 20$, +0.09). It pays a price of 0.030 F1 in the grid setting (0.911 against 0.941). On real candidates, however, it is *worse* than C2 (F1 0.784 ± 0.016 against 0.828 ± 0.002; every one of its five seeds, 0.765–0.808, is below every C2 seed, 0.826–0.830), mainly through false positives on unexpressed slots (1.1% against 0.4%). Hypothesis H3 is therefore not supported with generic augmentation. The diagnosis (Section 5.2) is that real lists are short and that their trap is the value of another slot of the same utterance, which the easy distractors of v1 never show.

Confusion-oriented augmentation: robust without cost C3 v2 replaces the easy distractors by the `confusion` preset and over-samples short lists. Table 12 compares it with C2 on the MultiATIS++ English test set. On real lists the two are indistinguishable (0.829 against 0.828; no significant difference, family 2), and the price in the grid setting falls to 0.007 F1 (not significant after Holm). Outside the training setting of C2, C3 keeps the robustness of the augmentation: at $k = 20$ without missing values its F1 is 0.097 higher than C2's, with a much higher precision (0.938 against 0.749); at $k = 10$, $p_{\text{absent}} = 0.5$, it is 0.062 higher and abstains correctly when the gold value is missing in 98% of the cases, against 67% for C2. C3 v2 is also more stable across seeds than v1 (standard deviations 0.003–0.012 against 0.008–0.018). Across the six dataset–language pairs of the grid (family 2, 36 tests), C3 and C2 differ by less than 0.01 F1 except on MultiATIS++ French with real lists (−0.022); four differences are significant after Holm, this one and three small ones of opposite signs (MTOP French with oracle lists, −0.007; Multi3WOZ English with oracle lists, +0.006, and with real lists, −0.008); C3 has slightly fewer false positives with oracle lists and slightly more with real lists, and a lower choice accuracy with oracle lists (by 0.007 to 0.017).

Revised hypothesis H3 Random augmentation of the option lists makes the judge robust to changes of context *only if it reproduces the confusions of real proposers* (values of other slots, short lists); generic random distractors make it robust on synthetic sweeps but hurt it on real lists. The two versions together constitute an ablation of the augmentation design; finer ablations (one component at a time) and schema perturbations in the spirit of SGD-X are left for future work.

6. Analysis and discussion

6.1. What the errors tell us

The decomposition of Section 3.1 separates three failure modes that a single F1 score conflates, and the three judges fail in different ways.

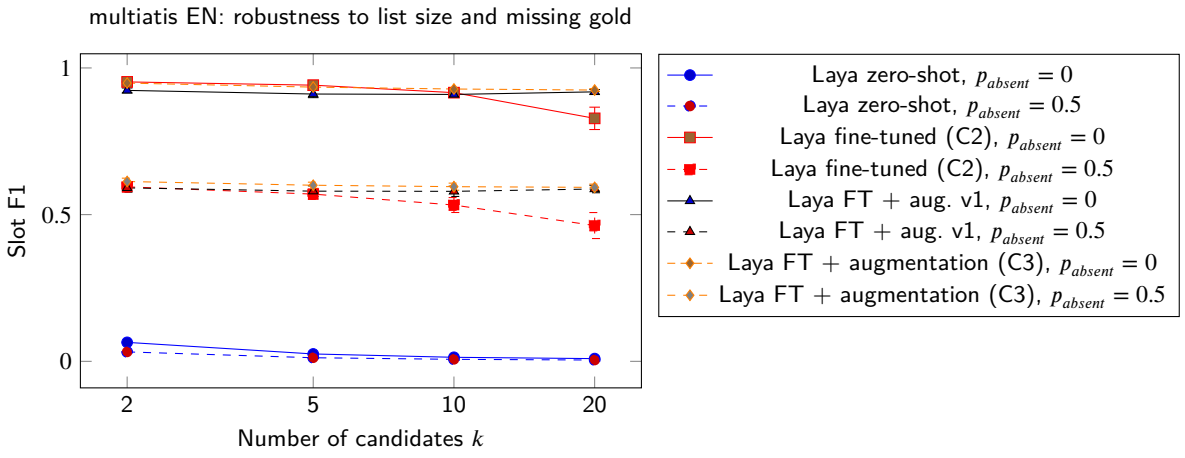


Figure 5: Robustness to the number of candidates k and to a missing gold value ($p_{\text{absent}} = 0.5$, dashed), MultiATIS++ English test set, mean ± s.d. over seeds, for zero-shot Laya, C2 and the two augmentation designs.

Zero-shot Laya lacks the information, not the decision rule Before fine-tuning, Laya's errors were dominated by false positives and by preferences for n-grams that contain the gold value ("on april" for "april", "flight from charlotte"), type confusions ("april" proposed for a day number or a city) and occasional wrong abstentions; its probabilities stayed close to 0.5 even when it was right. No threshold or calibration could repair this: on the MultiATIS++ English test set, it answers on 83% of the unexpressed slots and chooses the right value in only 20% of the cases where it is present. The model had to learn what "this slot is expressed in the last message" means for this task.

LLM judges choose well but do not abstain On MultiATIS++, the LLM judges choose the right value as often as the fine-tuned judge when it is present, but they fill 15–21% of the slots that the user did not express. Their errors are mostly over-eager answers: a value of the right type is present in the list, often one that the user did express for *another* slot, and the model selects it despite the explicit "not specified" option and the instruction to use it. Without in-domain supervision, the notion of "expressed in the last message" is underspecified for a general model, and the explicit option does not suffice to calibrate it. Whether a few in-context examples change this behaviour remains open (Section 6.6).

The pipeline's bottleneck is the proposers With real candidates, recall errors dominate (Section 5.2), especially in French, where the generic proposers (lexicon, GLiNER with English-like slot descriptions) are weaker. This is good news for the architecture: errors are attributable, and the cheapest components to improve are the proposers, which can be changed without retraining the judge.

Judge or tagger? The sequence tagger (Section 5.3) sets the limit of our claims. When values are literal spans of the utterance, as on MultiATIS++ and MTOP, a tagger trained on the same data is more accurate than the pipeline with generic proposers, and cheaper per utterance. The proposer–judge architecture pays off when values are *not* literal spans: references to the dialogue history, values of the context, and canonical values that differ from the words of the user, which are frequent in French Multi3WOZ. The two are complementary rather than competing: the tagger is a strong proposer, and the judge adds what the tagger lacks, namely the choice among sources, abstention and a calibrated confidence for deciding whether to act.

6.2. Cost

The fine-tuned judge is cheaper in every respect that we can measure. It needs a fraction of one GPU (below 2 GiB of memory), against one or two full H200 GPUs for the LLMs; it answers a single request 3 to 6 times faster and processes 5 to 38 times more requests per second in batches. Its training is cheap: minutes for single-turn datasets and about one hour per seed for the largest (bilingual, multi-turn) training set on one GPU. The request-level ratio is smaller than we expected (H1 assumed 10): an LLM that generates a single token is fast on an H200, and our Laya implementation was not optimised for latency (no compilation, no padding to fixed lengths). The memory footprint, which lets the judge share a GPU with the proposers, is the more decisive advantage for deployment. Fine-tuning the

LLMs themselves on the same items would likely close part of the accuracy gap (Aguirre et al., 2025), but at a training and serving cost that is precisely what the small judge avoids; we leave this comparison out of scope.

6.3. Why the small judge is better calibrated

Two features of the fine-tuned judge plausibly explain its better risk–coverage behaviour. First, it is trained with a strictly proper scoring rule on exactly the decision it makes at inference, with "not specified" as an explicit, frequently correct option; the log score rewards honest probabilities, including for abstention (Gneiting and Raftery, 2007). Second, the decision is read in one forward pass from a head designed for option probabilities, whereas the LLMs' first-token distribution is a by-product of generative training, flattened or sharpened by instruction tuning. Even so, the small judge is over-confident (T up to 4.5) and increasingly so away from its training distribution, so temperature scaling on development data remains necessary. The dominant factor for selective decision is, however, not the scale but the *ranking*: temperature scaling does not change the order of the requests by confidence, and the LLMs' AURC stays high after scaling because their confident answers are often wrong. Recalibration methods designed for LLM judges, such as linear probes on hidden states (Radharapu et al., 2026) or prediction-distribution correction (Li et al., 2025a), may narrow this gap and are a natural next comparison.

6.4. Implications for hybrid systems

The three-zone decision turns the judge into a component that knows when to defer. With real candidates, the fine-tuned judge decides 39–68% of the MTOP requests and nearly all MultiATIS++ and Multi3WOZ requests on its own at a 5% target risk; the remainder can be asked back to the user or sent to a costlier component. A natural design is a cascade in which the small judge handles confident cases and an LLM the uncertain ones (Lee et al., 2024). Our results suggest caution: the LLM judges are *less* reliable than the small judge on the hard cases too, and Tikhomirov and Savchenko (2026) report that an LLM verifier placed behind encoder confidence gates does not improve the cost–quality frontier because of a selection effect. Asking the user may thus be a better use of uncertainty than escalating to an LLM; measuring such a cascade directly is left for future work.

6.5. Application domains

The experiments cover three kinds of applications: air-travel information (MultiATIS++), personal assistants over eleven domains (MTOP: alarm, calling, event, messaging, music, news, people, recipes, reminder, timer, weather) and multi-domain task-oriented dialogue for tourism services (Multi3WOZ: restaurant, hotel, attraction, train, taxi), in English and French. The mechanism we study is, however, not specific to slot filling: a compact judge that selects one of several proposed options or abstains, trained with a proper scoring rule and calibrated so that its confidence can drive a three-zone decision. It plausibly transfers to other frequent, well-defined decisions for which supervision can be collected, which we have not evaluated: selecting the value of a form field or of a structured record among the outputs of extractors (e-mails, tickets, reports); selecting a tool, or the value of a tool argument, in an agent, with an explicit decision to ask for confirmation; and replacing LLM judges that arbitrate between options inside a pipeline, at a fraction of their cost and memory footprint, which also makes on-premise deployment easier. The selective decision matters most where a wrong action costs more than a question to the user, as in health, finance, public administration or bookings. Two conditions learnt from this study apply in every case: in-domain (and in-language) training data are needed, and the proposers must be strong, since the judge cannot recover a value that is not proposed and a dedicated extractor may otherwise be the better choice (Section 5.3).

6.6. Limitations

- **Single test pass, development-set decisions.** All protocol decisions were taken on development data, and checkpoint selection used the first 300 development examples of each dataset, which also served for those decisions; the test sets were then used once, under a protocol frozen beforehand. Latencies and the descriptive statistics of the candidate lists come from development data.

- **Gold intent and domain.** Requests use the gold intent (or the gold domain on Multi3WOZ), which isolates slot filling from intent detection. The domain filter also leaves out 1.9% of the Multi3WOZ gold pairs, which caps recall.

- **Scope.** H3 is supported only for the confusion-oriented augmentation, and the sweeps and the `hard` ablation were run for the Laya judges on MultiATIS++ English only. The latency part of H1 is not supported (ratio 3

to 6 instead of 10). We did not study the two-step decision (H4), unseen-domain splits, renamed or paraphrased schemas, a generative LLM extractor without proposers, or a cascade with deferral to an LLM; and we did not add the tagger's spans to the real candidate lists, although Section 5.3 suggests that it is the most promising change.

- **LLM coverage.** Three LLMs from two families were evaluated, zero-shot only, with one prompt; a third family (Mistral-Small-4) could not be served with our vLLM version. Prompt engineering, few-shot prompting or fine-tuning might improve the LLMs.
- **Languages and data.** Only English and French are covered, on public benchmarks that approximate real assistant traffic. MultiATIS++ is narrow (one domain) and cannot be redistributed (see the Data and code availability statement).
- **Bilingual training.** Joint English–French training is unstable with real candidates on Multi3WOZ (wrong abstentions, Section 5.4), probably because the checkpoint is selected on English oracle lists; we therefore use monolingual models.
- **Training objective.** We fine-tune with a supervised log score because Laya's original reinforcement-learning training code is not released; the comparison with the original procedure is not possible. With full labels and calibrated probabilities, a threshold on the probability is already the optimal decision rule, so reinforcement learning would add little to the task as posed here. It becomes relevant in three settings that we did not study: (i) learning the act/ignore/ask policy directly under asymmetric costs, for instance by adding the expected reward $\sum_a p(a \mid q)\, R(a, y^*)$ of the decision to the log score (exact, since the action set is small); (ii) learning from user feedback (confirmations, corrections) instead of gold labels, as an off-policy contextual bandit; and (iii) learning when to ask at the level of a whole dialogue, with a reward that trades task success against the number of questions and a simulated user, e.g. on Multi3WOZ. In each case calibration must be monitored, since a reward objective is not a proper scoring rule.

6.7. Threats to validity

Internal validity (i) *Test-set exposure.* The MultiATIS++ English test split was opened several times during early development; results from these runs are labelled exploratory and none is used for the main claims; all later decisions were taken on development data, and the test sets will be used once per frozen configuration. (ii) *Selection criterion.* The checkpoint criterion was changed twice on development data (global loss, then a class-balanced loss, then development F1), because over-confidence distorted the losses: one C3 seed kept an epoch with a lower loss but 663 test false positives against 304–462 for the other seeds. (iii) *Configuration drift.* An early bug (oracle generators not fitted on training data) produced empty or impoverished candidate lists and invalidated the first runs; another series ran on stale code. Both were detected, the affected runs were re-labelled or discarded, and a guard and a code fingerprint in every run now prevent recurrence. (iv) *Latency* was measured on 100 development examples per dataset with batches of size one; it depends on the implementation (inference engine, compilation, padding), which favours the LLMs, served by an optimised engine.

External validity Public benchmarks are cleaner than real traffic; annotation conventions (span boundaries, nested MTOP slots, Multi3WOZ state differences) shape what counts as correct. Our hand-written slot descriptions were frozen before evaluation, but other descriptions could favour one judge or the other.

Construct validity With most slots unexpressed, slot F1 mostly measures abstention; we therefore always report choice accuracy, false-positive rate and abstention when the gold value is absent alongside F1, and frame accuracy as a stricter view. Joint goal accuracy for Multi3WOZ is not reported (Section 4.4), so our Multi3WOZ numbers are not comparable with dialogue-state-tracking leaderboards.

Contamination The benchmarks are public and may be part of the LLMs' training data. Contamination would favour the LLMs, so it cannot explain their lower scores; it could however mask an even larger gap. We have no unseen-domain control.

Statistical validity Training seeds are few (three to five); the LLM judges decode greedily, so their three runs per cell differ only through the seed of the candidate lists. Paired tests compare seed-13 runs: intervals and p-values reflect test-sample variability, and seed variability is reported separately. The Holm correction is applied within families of comparisons, not across the whole paper.

7. Conclusion

We studied a hybrid architecture in which inexpensive extractors propose candidate values and a judge selects one of them or abstains, and we asked whether a small decision model can play the judge better than large open-weight language models. On three benchmarks, two languages and single- and multi-turn inputs, a fine-tuned Laya judge with fewer than half a billion parameters was more accurate than LLM judges of 27 to 180 billion parameters, 3 to 6 times faster per request and 5 to 38 times faster in batch throughput, on a fraction of one GPU. The main reason is abstention: the LLM judges choose well when the right value is present, but too often answer when the user expressed nothing. After calibration, the small judge's confidences also rank its errors far better, which lets a system decide most requests on its own at a few percent risk and ask back on the rest; the LLM judges cannot do so on the hardest dataset. Two design lessons emerge. Robustness to the shape of the option lists comes from training-time augmentation only when the augmentation reproduces the confusions of real proposers. And cross-lingual transfer remains imperfect: zero-shot transfer from English costs 0.04–0.09 F1, and joint English–French training, which helps with oracle lists, becomes unstable with real candidates unless the checkpoint is selected on matching data. With real candidates, the bottleneck moves to the proposers: a fine-tuned sequence tagger alone outperforms our pipeline when values are literal spans of the utterance, while the pipeline wins when values come from the dialogue history or must be mapped to canonical forms. The judge is therefore best seen not as a competitor of extraction models but as the decision layer on top of them.

Beyond slot filling, these results support a general design principle for agentic and dialogue systems: when a decision is frequent, well defined and can be supervised, a compact model trained with a proper scoring rule on the exact decision format, and calibrated on held-out data, can be more accurate, cheaper and more trustworthy than prompting a general model. The next steps are to use the tagger as one of the proposers, to learn the decision policy from asymmetric costs and from user feedback with reinforcement learning, at the level of single requests and of whole dialogues, and to study few-shot LLM judges, the two-step decision, unseen domains and robustness to schema changes.

Data and code availability

The code of this study (data conversion, candidate generators, judges, fine-tuning, decision module, statistics, tables and figures), all configurations and the scripts that reproduce every run are available at `https://github.com/ChristopheCruz/propose-then-judge-slots` under the MIT licence. The converted splits of MTOP (Li et al., 2021) will be released under CC BY-SA 4.0, as required by the licence of the original data (attribution, indication of the changes, same licence), and those of Multi3WOZ (Hu et al., 2023) under the MIT licence of the original data, together with the candidate lists and the predictions of all judges. MultiATIS++ is distributed by the Linguistic Data Consortium (*ATIS – Seven Languages*, LDC2021T04) (Mansour and Haider, 2021) under a licence that does not allow redistribution; for this dataset we release only the conversion scripts and the SHA-256 hashes of the converted files, so that holders of the LDC release can regenerate them exactly, and predictions without text. The fine-tuned checkpoints for MTOP and Multi3WOZ will be released on GitHub. Exact model revisions are listed in Table 3 and in the released `MODELES.lock.json`.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the French National Research Agency (ANR) through the LabCom LAMAE project, grant number ANR-24-LCV1-0008, and the CAIRE project, grant number ANR-23-CMAS-0031, which funded the computing infrastructure used for all experiments.

CRediT authorship contribution statement

Christophe Cruz: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration, Funding acquisition. **Hussam Ghanem:** Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Hocine Cherifi:** Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Claude, an AI assistant developed by Anthropic, in order to assist with software development (experiment code, tests and scripts), to check bibliographic references against their sources, and to prepare drafts of the documentation and of this manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

The experiments were run on a server with two NVIDIA H200 NVL GPUs funded by the CAIRE project (ANR-23-CMAS-0031). We thank the Linguistic Data Consortium (LDC) for granting us access to the MultiATIS++ corpus (LDC2021T04).

References

Aguirre, M., Méndez, A., del Pozo, A., Torres, M.I., Torralbo, M., 2025. Fine-tuning medium-scale LLMs for joint intent classification and slot filling: A data-efficient and cost-effective solution for SMEs, in: Proceedings of the 31st International Conference on Computational Linguistics: Industry Track, Association for Computational Linguistics, Abu Dhabi, UAE. pp. 251–262. URL: https://aclanthology.org/2025.coling-industry.21/.

Chen, Q., Zhuo, Z., Wang, W., 2019. BERT for joint intent classification and slot filling. URL: https://arxiv.org/abs/1902.10909, doi:10.48550/arXiv.1902.10909, arXiv:1902.10909. preprint.

Convai Innovations, 2026. Laya: Multilingual, non-autoregressive system 1 decision model. Hugging Face model card, https://huggingface.co/convaiinnovations/laya, revision 7b928d828b7b0e022f929d9bd2e44165aa270148; code: https://github.com/NandhaKishorM/laya. Model card and software, not peer-reviewed; no technical report linked. Accessed 2026-10-30.

FitzGerald, J., Hench, C., Peris, C., Mackie, S., Rottmann, K., Sanchez, A., Nash, A., Urbach, L., Kakarala, V., Singh, R., Ranganath, S., Crist, L., Britan, M., Leeuwis, W., Tur, G., Natarajan, P., 2023. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Toronto, Canada. pp. 4277–4302. URL: https://aclanthology.org/2023.acl-long.235/, doi:10.18653/v1/2023.acl-long.235.

Geifman, Y., El-Yaniv, R., 2017. Selective classification for deep neural networks, in: Guyon, I., Von Luxburg, U., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), Advances in Neural Information Processing Systems 30, Curran Associates, Inc.. pp. 4878–4887. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

Gemma Team, 2026. Gemma 4 technical report. URL: https://arxiv.org/abs/2607.02770, arXiv:2607.02770. preprint.

Gneiting, T., Raftery, A.E., 2007. Strictly proper scoring rules, prediction, and estimation. Journal of the American Statistical Association 102, 359–378. doi:10.1198/016214506000001437.

Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning, PMLR. pp. 1321–1330. URL: https://proceedings.mlr.press/v70/guo17a.html.

Heck, M., Lubis, N., Ruppik, B., Vukovic, R., Feng, S., Geishauser, C., Lin, H.c., van Niekerk, C., Gašić, M., 2023. ChatGPT for zero-shot dialogue state tracking: A solution or an opportunity?, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Association for Computational Linguistics, Toronto, Canada. pp. 936–950. URL: https://aclanthology.org/2023.acl-short.81/, doi:10.18653/v1/2023.acl-short.81.

Hu, S., Zhou, H., Hergul, M., Gritta, M., Zhang, G., Iacobacci, I., Vulić, I., Korhonen, A., 2023. Multi^{3}WOZ: A multilingual, multi-domain, multi-parallel dataset for training and evaluating culturally adapted task-oriented dialog systems. Transactions of the Association for Computational Linguistics 11, 1396–1415. URL: https://aclanthology.org/2023.tacl-1.79/, doi:10.1162/tacl_a_00609.

Hu, Y., Lee, C.H., Xie, T., Yu, T., Smith, N.A., Ostendorf, M., 2022. In-context learning for few-shot dialogue state tracking, in: Findings of the Association for Computational Linguistics: EMNLP 2022, Association for Computational Linguistics, Abu Dhabi, United Arab Emirates. pp. 2627–2643. URL: https://aclanthology.org/2022.findings-emnlp.193/, doi:10.18653/v1/2022.findings-emnlp.193.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), ACM. pp. 611–626. doi:10.1145/3600006.3613165.

Lee, C.H., Cheng, H., Ostendorf, M., 2024. OrchestraLLM: Efficient orchestration of language models for dialogue state tracking, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Mexico City, Mexico. pp. 1434–1445. URL: https://aclanthology.org/2024.naacl-long.79/, doi:10.18653/v1/2024.naacl-long.79.

Lee, H., Gupta, R., Rastogi, A., Cao, Y., Zhang, B., Wu, Y., 2022. SGD-X: A benchmark for robust generalization in schema-guided dialogue systems, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 10938–10946. URL: https://ojs.aaai.org/index.php/AAAI/article/view/21341, doi:10.1609/aaai.v36i10.21341.

Lee, H., Lee, J., Kim, T.Y., 2019. SUMBT: Slot-utterance matching for universal and scalable belief tracking, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy. pp. 5478–5483. URL: https://aclanthology.org/P19-1546/, doi:10.18653/v1/P19-1546.

Li, H., Arora, A., Chen, S., Gupta, A., Gupta, S., Mehdad, Y., 2021. MTOP: A comprehensive multilingual task-oriented semantic parsing benchmark, in: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, Association for Computational Linguistics, Online. pp. 2950–2962. URL: https://aclanthology.org/2021.eacl-main.257/, doi:10.18653/v1/2021.eacl-main.257.

Li, H., Chen, J., Ai, Q., Chu, Z., Zhou, Y., Dong, Q., Liu, Y., 2025a. CalibraEval: Calibrating prediction distribution to mitigate selection bias in LLMs-as-judges, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 16537–16552. URL: https://aclanthology.org/2025.acl-long.808/, doi:10.18653/v1/2025.acl-long.808.

Li, W., Huang, J., Huang, P., Xu, Y., Fan, J., 2025b. Synergistic augmentation: Enhancing cross-domain zero-shot slot filling with small model-assisted large language models, in: Findings of the Association for Computational Linguistics: ACL 2025, Association for Computational Linguistics, Vienna, Austria. pp. 21299–21312. URL: https://aclanthology.org/2025.findings-acl.1097/, doi:10.18653/v1/2025.findings-acl.1097.

Li, Z., Chen, Z.Z., Ross, M., Huber, P., Moon, S., Lin, Z., Dong, L., Sagar, A., Yan, X., Crook, P.A., 2024. Large language models as zero-shot dialogue state tracker through function calling, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 8688–8704. URL: https://aclanthology.org/2024.acl-long.471/, doi:10.18653/v1/2024.acl-long.471.

Lin, Y., Xiao, Z., Natarajan, K., Sankaranarayanan, M., Nawanit, N., Parashar, R., Zhang, A., Konaraddi, K., Mote, R., Niu, W., 2026. ReacTOD: Bounded neuro-symbolic agentic NLU for zero-shot dialogue state tracking, in: Proceedings of the 6th Workshop on Trustworthy NLP (TrustNLP 2026), Association for Computational Linguistics, San Diego, California. pp. 342–352. URL: https://aclanthology.org/2026.trustnlp-main.21/, doi:10.18653/v1/2026.trustnlp-main.21.

Lin, Z., Liu, B., Moon, S., Crook, P., Zhou, Z., Wang, Z., Yu, Z., Madotto, A., Cho, E., Subba, R., 2021. Leveraging slot descriptions for zero-shot cross-domain dialogue StateTracking, in: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, Online. pp. 5640–5648. URL: https://aclanthology.org/2021.naacl-main.448/, doi:10.18653/v1/2021.naacl-main.448.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., Zhu, C., 2023. G-eval: NLG evaluation using Gpt-4 with better human alignment, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore. pp. 2511–2522. URL: https://aclanthology.org/2023.emnlp-main.153/, doi:10.18653/v1/2023.emnlp-main.153.

Mansour, S., Haider, B., 2021. ATIS - seven languages. Linguistic Data Consortium, LDC2021T04, Web Download, Philadelphia. URL: https://catalog.ldc.upenn.edu/LDC2021T04, doi:10.35111/g9h5-0p74. [dataset].

Marone, M., Weller, O., Fleshman, W., Yang, E., Lawrie, D., Van Durme, B., 2026. mmBERT: A modern multilingual encoder with annealed language learning, in: Proceedings of the 43rd International Conference on Machine Learning, PMLR. pp. 86768–86783. URL: https://proceedings.mlr.press/v306/marone26a.html. preprint: arXiv:2509.06888.

Moradshahi, M., Shen, T., Bali, K., Choudhury, M., de Chalendar, G., Goel, A., Kim, S., Kodali, P., Kumaraguru, P., Semmar, N., Semnani, S., Seo, J., Seshadri, V., Shrivastava, M., Sun, M., Yadavalli, A., You, C., Xiong, D., Lam, M., 2023. X-RiSAWOZ: High-quality end-to-end multilingual dialogue datasets and few-shot agents, in: Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, Toronto, Canada. pp. 2773–2794. URL: https://aclanthology.org/2023.findings-acl.174/, doi:10.18653/v1/2023.findings-acl.174.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al., 2019. PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems 32 (NeurIPS 2019), Curran Associates. pp. 8024–8035.

Pecher, B., Srba, I., Bielikova, M., 2025. Comparing specialised small and general large language models on text classification: 100 labelled samples to achieve break-Even performance, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Suzhou, China. pp. 165–184. URL: https://aclanthology.org/2025.emnlp-main.9/, doi:10.18653/v1/2025.emnlp-main.9.

Qwen Team, 2026a. Qwen3.8-Flash-Next: A new architecture, towards ultimate cost-efficiency. URL: https://qwen.ai/blog?id=qwen3.8-flash-next. blog post; citation recommended by the Qwen/Qwen3.8-Flash-Next-FP8 model card. Accessed 2026-10-30.

Qwen Team, 2026b. Qwen3.8-Max: A new bar for coding and cowork. URL: https://qwen.ai/blog?id=qwen3.8. blog post; citation recommended by the Qwen/Qwen3.8-27B model card. Accessed 2026-10-30.

Radharapu, B., Saxena, E., Li, K., Whitehouse, C., Williams, A., Cancedda, N., 2026. Calibrating LLM judges: Linear probes for fast and reliable uncertainty estimation, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track), Association for Computational Linguistics, San Diego, California, USA. pp. 193–216. URL: https://aclanthology.org/2026.acl-industry.14/, doi:10.18653/v1/2026.acl-industry.14.

Rana, M., Hacioglu, K., Gopalan, S., Boothalingam, M., 2025. Zero-shot slot filling in the age of LLMs for dialogue systems, in: Proceedings of the 31st International Conference on Computational Linguistics: Industry Track, Association for Computational Linguistics, Abu Dhabi, UAE.

pp. 697–706. URL: https://aclanthology.org/2025.coling-industry.59/.

Rastogi, A., Zang, X., Sunkara, S., Gupta, R., Khaitan, P., 2020. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 8689–8696. URL: https://ojs.aaai.org/index.php/AAAI/article/view/6394, doi:10.1609/aaai.v34i05.6394.

Tikhomirov, D., Savchenko, A., 2026. The selection effect: Why compact LLM verifiers for emotion classification fail behind encoder confidence gates, in: Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Budapest, Hungary. pp. 24095–24114. URL: https://aclanthology.org/2026.emnlp-main.1049/, doi:10.18653/2026.emnlp-main.1049.

Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., Sui, Z., 2024. Large language models are not fair evaluators, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 9440–9450. URL: https://aclanthology.org/2024.acl-long.511/, doi:10.18653/v1/2024.acl-long.511.

Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Adams, G.T., Howard, J., Poli, I., 2025. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 2526–2547. URL: https://aclanthology.org/2025.acl-long.127/, doi:10.18653/v1/2025.acl-long.127.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., et al., 2020. Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics. pp. 38–45. doi:10.18653/v1/2020.emnlp-demos.6.

Wu, C.S., Madotto, A., Hosseini-Asl, E., Xiong, C., Socher, R., Fung, P., 2019. Transferable multi-domain state generator for task-oriented dialogue systems, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy. pp. 808–819. URL: https://aclanthology.org/P19-1078/, doi:10.18653/v1/P19-1078.

Wu, L., Petroni, F., Josifoski, M., Riedel, S., Zettlemoyer, L., 2020. Scalable zero-shot entity linking with dense entity retrieval, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online. pp. 6397–6407. URL: https://aclanthology.org/2020.emnlp-main.519/, doi:10.18653/v1/2020.emnlp-main.519.

Xu, W., Haider, B., Mansour, S., 2020. End-to-end slot alignment and recognition for cross-lingual NLU, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online. pp. 5052–5063. URL: https://aclanthology.org/2020.emnlp-main.410/, doi:10.18653/v1/2020.emnlp-main.410.

Yin, W., Hay, J., Roth, D., 2019. Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Association for Computational Linguistics, Hong Kong, China. pp. 3914–3923. URL: https://aclanthology.org/D19-1404/, doi:10.18653/v1/D19-1404.

Zadrozny, B., Elkan, C., 2002. Transforming classifier scores into accurate multiclass probability estimates, in: Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM. pp. 694–699. doi:10.1145/775047.775151.

Zaratiana, U., Tomeh, N., Holat, P., Charnois, T., 2024. GLiNER: Generalist model for named entity recognition using bidirectional transformer, in: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Mexico City, Mexico. pp. 5364–5376. URL: https://aclanthology.org/2024.naacl-long.300/, doi:10.18653/v1/2024.naacl-long.300.

Zhang, J., Hashimoto, K., Wu, C.S., Wang, Y., Yu, P., Socher, R., Xiong, C., 2020. Find or classify? dual strategy for slot-value predictions on multi-domain dialog state tracking, in: Proceedings of the Ninth Joint Conference on Lexical and Computational Semantics, Association for Computational Linguistics, Barcelona, Spain (Online). pp. 154–167. URL: https://aclanthology.org/2020.starsem-1.17/.

Zhao, J., Gupta, R., Cao, Y., Yu, D., Wang, M., Lee, H., Rastogi, A., Shafran, I., Wu, Y., 2022. Description-driven task-oriented dialog modeling. URL: https://arxiv.org/abs/2201.08904, doi:10.48550/arXiv.2201.08904, arXiv:2201.08904. preprint.

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J., Stoica, I., 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (Eds.), Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track), Curran Associates, Inc.. pp. 46595–46623. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html, doi:10.52202/075280-2020.

A. Hyperparameters

Table 13 lists the settings of fine-tuning (`configs/train/c2.yaml`, `c3.yaml`, `c3v2.yaml`) and evaluation. They were fixed on development data and frozen at the end of the first phase (26 October 2026).

Table 14 gives the amount of training data. Every training turn yields one question per slot of its intent; all questions whose slot is expressed are kept, and at most two questions with an unexpressed slot per expressed one, drawn at random (`unexpressed_ratio` = 2), because unexpressed slots are by far the most frequent. C2 and C3 use the same questions (C3 only changes the candidate lists), for three epochs; the checkpoint is selected on the first 300 development examples. The tagger C0 is trained on the same turns, with one BIO label per word, for ten epochs, and selected on the whole development set.

B. Prompt of the LLM judge and questions asked to Laya

B.1. LLM judge (prompt `slot-judge-v1`)

System message

```
You are a slot-filling judge for a task-oriented dialogue system. You answer
with a single capital letter and nothing else.
```

User message Fields in braces are filled per request; `{options}` lists the candidates as "`A. value`", "`B. value`"..., followed by the "not specified" option. Empty fields are rendered as `unknown` (intent), `none` (filled slots) and `(none)` (history).

```
Task: choose the value that the user's last message gives for the slot below.
Pick the candidate that the user expresses or clearly refers to in the last
message (the history and the slots already filled are context only). If the
last message gives no value for this slot, or none of the candidates is right,
pick the "not specified" option.
The conversation may be in a language other than English; candidates are
written as they appear.

Slot: {slot}
Slot description: {description}
Current intent: {intent}
Slots already filled: {filled}
Conversation history:
{history}

User's last message ({lang}): {utterance}

Candidates:
{options}

Answer with the letter of one candidate only.
```

(Line breaks inside paragraphs are added here for the page width; in the code each paragraph is one line.)

"Not specified" option

```
not specified (the last message gives no value for this slot)
```

Few-shot format (not used in the reported runs) Each shot is the same user message followed by an assistant turn containing the letter; in plain-completion mode the answer is rendered as `Answer: {letter}`.

Reading the answer One token is generated at temperature 0; the top-20 log-probabilities of the first token are read; the probability masses of the tokens that reduce to an option letter (after removing the SentencePiece and byte-level space markers and trailing "`.`", "`)`", "`:`") are summed and renormalised over the offered letters. Reasoning is disabled with `enable_thinking: false` for the Qwen3.8 and Gemma 4 chat templates.

B.2. Laya questions

Single mode (main condition) Question of type `choice`; options are "`A: <candidate>`", "`B: <candidate>`"..., then the "not specified" option.

```
Which value does the user's last message give for the slot "{slot}"
({description})?
```

"Not specified" option:

```
not specified: the last message gives no value for this slot
```

Two-step mode (implemented, not evaluated in this study) First question ("is the slot expressed?"), of type `noul` (true/false) or, as a fallback against the bias of that question type, `choice` with neutral keys A/B:

```
Does the user's last message give a value for the slot "{slot}"
({description})?
  A / true:  yes, the last message gives a value for this slot
  B / false: no, the last message gives no value for this slot
```

Second question, over the candidates only (no "not specified" option), in the same forward pass:

```
The user's last message gives a value for the slot "{slot}" ({description}).
Which one?
```

The final distribution is $p(c_i) = P(\text{expressed}) \cdot p(c_i \mid \text{expressed})$ and $p(\perp) = 1 - P(\text{expressed})$; below a threshold on $P(\text{expressed})$ (0.5 by default, to be tuned on development data) the judge answers "not specified".

State A dictionary with the fields `last_user_message`, `intent`, `slots_already_filled` (JSON, multi-turn only) and `history` ("`speaker: text`" lines), in this order.

C. Dataset conversion details

All loaders (`laya_slots/data/`) produce one record per turn with identifiers (dataset, dialogue, turn, language, split), the utterance, the history (Multi3WOZ), the schema available at request time (intents and slots with a description), and the gold intent, slot values (normalised value, surface form and character offsets) and cumulative state. Each prepared split is accompanied by a manifest with the source version and the SHA-256 of the raw and produced files, without timestamp, so that two identical preparations produce identical files.

Value normalisation The same deterministic function is applied to gold values and candidates before comparison: Unicode NFKC, typographic apostrophes, lower case, compacted spaces, edge punctuation removed; numbers written in words converted to digits from 0 to 99 in English and French (French "un"/"une" only when they are the whole value; ordinals only for date slots); times converted to `HH:MM` ("5 pm", "five thirty pm", "17h30", "midi"…; three- or four-digit values read as military time only for time slots). Relative dates are resolved only when a reference date is given (not used here).

MultiATIS++ TSV files of the LDC release (`train_XX.tsv`, `test_XX.tsv`) with BIO labels; BIO is converted to spans with the tolerant rule of `conlleval` (an orphan `I-x` opens a span), then to character offsets. The release has no development split: the 500 training identifiers with the smallest SHA-256 form the development set (4,478 / 500 / 893), identical in English and French. Multi-intent labels ("`atis_flight#atis_airfare`") keep the first intent as main intent; the full list is stored for an official intent accuracy. Slot descriptions are composed from prefix and attribute ("fromloc" + "city_name" → "departure city"); closed values for categorical attributes (day, month, period, class…) are collected on the training split only. Thirteen French training lines with inconsistent label counts are skipped.

MTOP Version 2 of 25 January 2021 (CC BY-SA 4.0). Top-level slots from the flat slot column (byte offsets converted to character offsets); a nested slot becomes one top-level slot whose value is the covered text, its sub-tree being kept in the metadata for a separate analysis. Official intent and slot names are kept (`IN:`/`SL:` prefixes). Slot descriptions written by hand in English, intent descriptions derived from the names; closed values for a few categorical slots, from training data only.

Multi3WOZ Corrected release of 15 December 2023 (MIT). One example per user turn; history = all previous turns; state before the turn = cumulative state after the previous user turn; slots of the turn = state difference. Five domains (hotel, train, restaurant, attraction, taxi), 30 slots named "domain-slot" as in `slot_descriptions.json`; "not mentioned", "none" and empty values mean absent, "dontcare" is an explicit value. Spans are recovered from the turn's span annotations, otherwise by string search; values without span are labelled as coming from the utterance, the history (accepted system offer) or the context. The domain of a turn is the majority domain of its slots, otherwise that of its dialogue acts. Validation and test lists are those of MultiWOZ (860 dialogues of each present in the data).

Licences and versions

Software All runs use the following versions, locked in `uv.lock`: the Laya runtime 0.3.22 (Convai Innovations, 2026), vLLM 0.30.0 (Kwon et al., 2023), GLiNER 0.2.29 (Zaratiana et al., 2024), PyTorch 2.13.0 (Paszke, Gross, Massa, Lerer, Bradbury, Chanan et al., 2019) and transformers 5.16.1 (Wolf, Debut, Sanh, Chaumond, Delangue, Moi, Cistac, Rault, Louf, Funtowicz, Davison, Shleifer et al., 2020), with Python 3.12.15.

Table 13
Hyperparameters of fine-tuning and evaluation.

Setting	Value
Fine-tuning (C2, C3)	
Initial checkpoint	`convaiinnovations/laya`, sub-folder `multilingual`, revision `7b928d8`
Mode	single question, "not specified" as last option
Training candidates (C2)	oracle, $k = 5$, $p_{\text{absent}} = 0$, `hard_no_overlap`
Augmentation (C3 v1)	$k \in \{2, 5, 10, 20\}$, $p_{\text{absent}} \in \{0, 0.2, 0.5\}$, distractors $\in$ {`easy`, `hard_no_overlap`}
Augmentation (C3 v2, "C3")	$k \in \{2, 2, 3, 3, 5, 10, 20\}$, $p_{\text{absent}} \in \{0, 0.2, 0.5\}$, distractors $\in$ {`confusion`, `hard_no_overlap`}
Maximum options at training	15 (so $k = 20$ is truncated to 15)
Unexpressed requests kept	at most 2 per expressed request and example (`unexpressed_ratio`)
Objective	log score (cross-entropy over the options of each question)
Epochs	3
Batch	32 questions, no gradient accumulation
Optimiser	AdamW; learning rate 2×10^{-5} (encoder), 10^{-4} (head); weight decay 0.01
Schedule	linear warm-up over 5% of steps, cosine decay
Other	gradient clipping 1.0; bf16 autocast over fp32 weights; gradient checkpointing (encoder and head)
Checkpoint selection	slot F1 on all requests of the first 300 development examples (`select_on: f1`)
Temperatures saved	neutral ($T = 1$); calibration fitted afterwards on development data
Seeds	13–15 (C2), 13–17 (C3)
Main configuration	monolingual (one model per dataset and language)
Bilingual training (analysis)	concatenation of EN and FR items; selection on the English development set, oracle candidates
Sequence tagger (C0, `configs/train/c0.yaml`)	
Encoder	`microsoft/mdeberta-v3-base`, revision `a048466`, weights from `refs/pr/5` (stored in fp16, loaded in fp32; bf16 autocast)
Labels	BIO over the slots seen with a span in training; label of a word on its first sub-word; words as in the candidate generators
Training	10 epochs, batch 32, AdamW, learning rate 3×10^{-5}, weight decay 0.01, linear warm-up over 10% of steps then linear decay, gradient clipping 1.0, maximum length 256 sub-words
Checkpoint selection	span F1 (exact slot and boundaries) on the whole development set
Inference	best-scoring span per slot (mean probability of its labels), normalised like the gold values; seeds 13–15
Evaluation	
Requests	all slots allowed by the gold intent (Multi3WOZ: gold domain); Multi3WOZ: history and state before the turn
Laya state	last message ($\leq$1,000 chars), intent, filled slots, last 4 turns ($\leq$1,500 chars)
Laya batch	128 states; option order shuffled with the run seed
Maximum options	15 (oracle grid), 20 (sweeps and real candidates); options $\leq$200 characters
Real candidates	lexicon, GLiNER (threshold 0.3), ontology (similarity $\geq$0.85), history; union $\leq$20, shuffled
LLM judges	vLLM 0.30.0, text only, context 4,096 tokens, 1 generated token, temperature 0, top-20 log-probabilities, reasoning off, batch 256, up to 25 options (oracle) or 20 (real)
Calibration	temperature (bounded search on $\log T$, $T \in [0.05, 20]$) and isotonic regression; ECE with 15 bins
Decision	target risks 2% and 5% (also 0.5% and 1% reported); two-fold cluster split of the development set
Bootstrap	1,000 cluster resamples, percentile 95% intervals, fixed seed
Permutation tests	10,000 cluster-wise permutations, two-sided, Holm correction per family

Table 14
Training data per dataset and language (main, monolingual configuration). Turns: training examples; questions: requests used to fine-tune Laya (C2 and C3); spans and labels: gold spans and BIO label set of the tagger C0; dev: development examples used for checkpoint selection.

			Laya (C2, C3)				Tagger (C0)		
Dataset	Lang.	Turns	Expressed	Unexpressed kept	Questions	Dev	Spans	Labels	Dev
MultiATIS++	EN	4,478	14,336	28,671	43,007	300	14,878	159	500
	FR	4,465	14,270	28,539	42,809	300	14,807	159	500
MTOP	EN	15,667	25,916	42,893	68,809	300	26,797	149	2,235
	FR	11,814	18,420	30,536	48,956	300	19,016	147	1,577
Multi3WOZ	EN	49,395	58,944	129,574	188,518	300	45,832	61	6,267
	FR	49,394	58,943	129,572	188,515	300	45,614	59	6,267

Table 15
Licences of the data and models used.

Resource	Licence	Redistribution of converted data
MultiATIS++ (LDC2021T04)	LDC user agreement (non-commercial research)	no (scripts and hashes only)
MTOP (v2)	CC BY-SA 4.0	yes, same licence
Multi3WOZ (15/12/2023)	MIT	yes
Laya (`7b928d8`)	Apache-2.0	–
GLiNER multi v2.1 (`443d26d`)	Apache-2.0	–
Qwen3.8-27B (`1d4bf0f`)	Apache-2.0	–
Qwen3.8-Flash-Next-FP8 (`236dfdf`)	Qwen Community License 1.0	–
Gemma-4-31B-it (`842da37`)	Apache-2.0	–